

习 题

1. 许多企业的计算机有 3 个不同的全球唯一标识符。试问它们分别是什么？
2. 在图 7-4 中，在 laserjet 后没有终结。试问这是为什么？
3. 考虑这样一种情况，一个网络恐怖主义分子同时破坏了全世界的所有 DNS 服务器。试问这将如何改变一个人使用 Internet 的能力？
4. DNS 使用 UDP 而没有采用 TCP。如果 DNS 数据包丢失，并且没有自动恢复机制。试问这会产生问题吗？如果会，它是如何解决呢？
5. John 希望有一个原始的域名，并使用一个随机程序来产生他的二级域名。他要在 com 通用域名下注册这个域名。生成的域名 253 个字符长。试问 com 注册处允许注册这个域名吗？
6. 试问只有一个 DNS 域名的机器可以有多个 IP 地址吗？这种情况是如何发生的？
7. 近年来，拥有 Web 站点的公司数量爆炸式增长。结果是，成千上万的公司注册在 com 域下，从而导致该域的顶级服务器负载极其繁重。请提出一种建议方案来缓解这个问题，同时又不用修改域名方案（也就是说，不引入新的顶级域名）。你的方案可以要求修改客户代码。
8. 有些电子邮件系统支持 Content Return: 头字段。它指定了当消息未被递交时是否返回邮件体。试问这个字段属于信封还是邮件头？
9. 电子邮件系统需要相应的目录，以便于查找人们的电子邮件地址。为了建立这样的目录，名字应该被分割成标准的组成部分（比如，第一个名字，最后一个名字）才有可能搜索。请讨论为了建立一个可接受的全球标准需要解决哪些问题。
10. 一个大型律师事务所有许多员工，每一位员工有一个电子邮件地址。每个员工的电子邮件地址是 <login>@lawfirm.com。然而，该事务所并没有明确定义登录格式。因此，有些员工使用他们的第一个名字作为自己的登录名，有些员工使用他们最后一个名字，还有些员工使用了名字的缩写，现在该事务所希望制定一个固定的格式，例如：
firstname.lastname@lawfirm.com
试问这个格式能用作全体员工的电子邮件地址吗？如何做才不至于造成太多混乱？
11. 一个二进制文件长 4560 个字节。如果使用基于 64 的编码技术，而且在每发出 110 字节以及结尾处插入 CR+LF。试问该文件将有多长？
12. 本书没有列出 5 个 MIME 类型。请检查你的浏览器或 Internet 来获取这方面的信息。
13. 假设你想给一个朋友发送一个 MP3 文件，但是你朋友的 ISP 限制入境邮件的大小为 1 MB，而该 MP3 文件是 4 MB。试问是否有办法通过使用 RFC 5322 和 MIME 来处理这种情形？
14. 假设约翰刚刚为他的工作电子邮件地址设置了自动转发机制，该转发机制接收他所有与业务相关的电子邮件，然后转发到他的私人电子邮件地址，他与他的妻子共享该私人邮箱。约翰的妻子不知道这一点，而且在他们的私人账户上触发了一个度假代理。因为约翰转发了他的电子邮件，因而他在他的工作机器上并没有设立度假守护进程。

试问，当约翰的工作电子邮件地址收到一封电子邮件时，会发生什么？

15. 任何一个标准都需要精确的语法，例如 RFC 5322，这样不同的实现才能相互协同工作。即使最简单的表项也必须小心定义。SMTP 头允许在 token 之间出现空白。请针对标记符之间的空白，给出两种似是而非的定义。
16. 度假代理是用户代理的一部分还是邮件传输代理的一部分？当然，它是通过用户代理设置的，它是用户代理实际发送的答复吗？请解释你的答案。
17. Chord 算法的一个简单版本被用在对等查询中。搜索不使用指取表，相反，它们被线性地绕成一个环，环可取任何一个方向。试问，一个节点能准确地预测它应该在哪个方向搜索吗？讨论你的答案。
18. IMAP 允许用户从远端邮箱获取和下载电子邮件。试问，这是否意味着内部邮箱格式必须标准化，因此客户端上的任何 IMAP 程序可以读取任何邮件服务器上的邮箱？讨论你的答案。
19. 考虑如图 7-71 所示的 Chord 环。假设节点 18 突然上线。试问图中的哪个指取表受到影响？如何受到影响的？
20. Webmail 使用了 POP3，还是 IMAP，或者什么都没有用？如果使用了其中之一，试问为什么会选择这个？如果两者都没用，实质上更接近哪个？
21. 当发送 Web 页面时，试问，它们会附加上 MIME 头吗？为什么？
22. 当用户使用 Firefox 点击一个链接时，一个特定的辅助应用程序就被启动；但在 IE 浏览器中点击相同的链接时，导致一个完全不同的辅助应用程序被启动，即使在这两种情况下返回的 MIME 类型是相同的。试问这种情况是否有可能发生？解释你的答案。
23. 虽然在课文中没有提到 URL 的另一种形式是使用 IP 地址，而不是它的 DNS 域名。请用此信息来解释为什么一个 DNS 域名不能以数字结尾。
24. 试想一下，在斯坦福大学数学系有人刚刚写了一个包括证明过程的新文档，他希望通过 FTP 将该文档分发给他的同事们审阅。他把程序放在 FTP 目录 ftp/pub/forReview/newProof.pdf。试问这一程序可能的 URL 是什么？
25. 在图 7-22 中，www.aportal.com 用一个 Cookie 跟踪记录用户的偏好。这项方案的缺点是 Cookie 被限制为 4 KB，因此如果要扩展偏好，例如，要增加关于许多股票、运动队、新闻报道的类型、多个城市的天气、在众多的产品类别特价商品和更多信息，可能会超出 4 KB 的限制。请设计另一种方案来保持跟踪用户的偏好但又没有这个问题。
26. 懒银行要为它的客户操作网络银行更加容易，因此，在客户签署并通过密码验证后，银行返回一个 Cookie，其中包含客户 ID 号。这样，客户在将来访问网上银行时可以不必识别自己，或者输入密码。试问，你认为这种想法怎么样？可行吗？它是一个好主意吗？
27. (a) 考虑如下的 HTML 标记：

```
<h1 title="this is the header"> HEADER 1 </h1>
```

在什么情况下，浏览器使用 TITLE 属性，如何使用的？

(b) TITLE 属性与 ALT 属性有什么不同？

28. 你如何在 HTML 中生成一个可点击的图像？举例说明。

29. 编写一个 HTML 页面。该页面包括一个邮件地址 `username@DomainName.com` 的链接。试问，当用户点击该链接时会发生什么？
30. 编写一个大学注册器。能列出多个学生，每个学生有一个姓名、地址和一个 GPA。
31. 对于下列应用程序，告诉它是否（1）有可能（2）最好使用 PHP 脚本或 JavaScript 中的哪一个，以及为什么：
 - (a) 显示从 1752 年 9 月以来的任何一个月的日历。
 - (b) 显示从阿姆斯特丹到纽约的航班时间表。
 - (c) 画出一个用户提供系数的多项式。
32. 写一个 JavaScript 程序，它接受一个大于 2 的整数，并告知它是否为一个素数。请注意，JavaScript 有 if 和 while 语句，这些语句的语法与 C 和 Java 相同。模运算符为 %。如果你需要 x 的平方根，使用 `Math.sqrt(x)`。
33. 一个 HTML 页面如下所示：

```
<html> <body>
<a href="www.info-source.com/welcome.html"> Click here for info </a>
</body> </html>
```

如果用户单击该超链，就打开了一个 TCP 连接，并且给服务器发送一系列的行。请列出发出的所有行。

34. If-Modified-Since 头可以用来检查一个缓存页面是否仍然有效。请求的页面可以包含图像、声音、视频等，以及 HTML。你认为这种技术用于 JPEG 图像相比 HTML 的效率更好还是更糟？仔细想想“效率”意味着什么，并解释你的答案。
35. 在当天的一个重大体育赛事中，比如一些流行运动的冠军赛，很多人都去访问官方网站。这个闪群与 2000 年佛罗里达州总统选举发生的闪群具有相同的意义吗？为什么是相同的？或者为什么不同呢？
36. 把单个 ISP 建设成 CDN，是否可行？如果可行，它将如何工作？如果不可行，这个想法错在哪里？
37. 假设不对语音 CD 进行压缩。为了播放两个小时的音乐，光盘必须包括多少 MB 的数据？
38. 在图 7-42(c)中，使用 4 位样值来表达 9 个信号值时会引入量化噪声。第一个样值，在 0 点是精确的，但后续几个就不准确了。对于在 1/32、2/32 以及 3/32 处的样值误差百分比分别是多少？
39. 一个伪随机模型能用来减少 Internet 电话所需的带宽吗？如果可以，必须满足什么条件（如果有的话）才能工作？如果不能，为什么？
40. 一个音频流服务器到一个媒体播放器的单向“距离”为 100 毫秒，它以 1 Mbps 速率输出。如果媒体播放器有一个 2 MB 的缓冲区，试问你能说出低水印标记和高水印标记的位置在哪里？
41. 试问 IP 语音遇到防火墙和流式音频遇到防火墙的问题相同吗？讨论你的答案。
42. 试问以每秒 50 帧、每个像素 16 位，传输未经压缩的 1200×800 像素的彩色帧所需要的比特率是多少？

43. 试问 MPEG 帧中 1 位的错误会影响该出错帧以外的其他帧吗？请解释你的答案。
44. 考虑一个有 50 000 个客户的视频服务器，每个客户每个月看三部电影。三分之二的电影是在晚上 9 时需求。服务器在这段时间内一次传输多少部电影？如果每部电影需要 6 Mbps，服务器需要多少个 OC-12 网络连接？
45. 假设齐普夫定律适用于对一个有 10 000 电影的 video 服务器的访问。如果服务器在内存中持有 1000 个最流行的电影，在磁盘上保持剩余的 9000 个电影，请给出所有引用都在内存中的比例表达式。写一个小程序，以便从数值上来评估该表达式。
46. 某些域名抢注行为已经注册了一些域名，这些域名是公共企业网站的拼写错误，例如，www.microsfot.com。请列出至少 5 个这样的域名。
47. 许多人注册了这样的 DNS 域名 www.word.com，其中 word 是一种常见字。对于以下每个类别，列出 5 个此类网站，并简要地总结它（例如，www.stomach.com 属于一个长岛的肠胃病）。下面是类别列表：动物、食品、家用物品和身体部位。对于最后一类，请坚持使用腰部以上的身体部位名称。
48. 重写图 6-6 的服务器，使它成为一个真实的使用 HTTP1.1 GET 命令可以访问的 Web 服务器。它也应该接受主机消息。服务器应该保持一份最近从磁盘读取的文件缓存，并且可能时用缓存来服务客户的请求。

第8章 网络安全

在最初几十年中，计算机网络主要被大学的研究人员用于发送电子邮件，以及被公司的员工用于共享打印机。在这样的条件下，安全问题并没有得到足够的关注。但现在，当成千上百万的普通市民利用网络来处理银行事务、网上购物和填报纳税时，出现了越来越多的安全问题，网络安全已成为一个巨大的问题。在本章中，我们将从几个角度来研究网络的安全问题，并指出各种安全陷阱，以及讨论许多能使网络更加安全的算法和协议。

安全性是一个范围很广阔的话题，同时也涉及大量的违法犯罪活动。在最简单的形式中，它关心的是如何保证好事者们不能读取或者（更糟的是）悄悄地修改给其他人的消息。它也涉及有人企图访问未经授权的远程服务。它还要处理这样的情形：如何判断一条自称来自 IRS（美国国税局）的“星期五之前付款”的消息确实来自 IRS，而并非来自黑手党（Mafia）。安全性也涉及合法消息被捕捉及重放的问题，以及人们企图否认自己曾经收到过某些特定的消息。

大多数安全问题都是由于某些恶意的人企图获得某种收益、引起别人关注，或者伤害他人而有意制造的。图 8-1 列出了一些最为常见的攻击者及目的。从这个列表中应该很清楚地看出，要想使一个网络变得更加安全，需要做的事情很多，而不仅仅只是改正编程错误就足够。这通常涉及与一些非常聪明、专一甚至还有足够资助的攻击者进行较量。同时也应该很清楚地看到，一些用来阻止随意攻击者的手段对于那些专业攻击者来说并没有太大的效果。警察的记录显示，多数的破坏性攻击并非是外人窃听电话线而是来自于内部积累的恩怨造成。安全系统在设计上应该考虑到这种情况。

攻击者	目的
学生	乐于窥探他人电子邮件
黑客	测试某人的安全系统；盗取数据
推销员	声称能代表整个欧洲，而不只是安道尔
公司	发现竞争者的策略性市场计划
离职员工	因被解雇而实施报复
会计	挪用公司公款
股票经纪人	拒绝通过E-mail向顾客做过的承诺
身份盗取	出售窃取信用卡号码
政府	了解敌人的军事或工业机密
恐怖分子	窃取生物战秘密

图 8-1 最常见的攻击者及目的

网络安全问题可以被粗略地分成 4 个相互交织的领域：保密、认证、不可否认和完整性控制。保密（secrecy）也称为机密（confidentiality），它的任务是确保信息不会被未经授权的用户访问。这也正是当人们考虑网络安全时首先想到的内容。认证是指当你在展示敏感信息或者进入商务交易之前你必须确定自己在跟谁通话。不可否认性涉及签名：你的

顾客下了一份要购买 1000 万个左手器具的订单，他事后声称每个器具的单价为 69 美分，你如何证明他原来定购的价格是 89 美分呢？或者他事后根本不承认自己曾经下过订单。最后，完整性控制是用来确定你收到的信息是真实的而不是被恶意攻击者在传输途中篡改过的伪造信息。

所有这些问题（保密、鉴别、不可否认和完整性控制）同样存在于传统的系统中，但是有一些重大的区别。在传统系统中，利用挂号信和文档上锁的做法可以获得完整性和保密性。现在，抢劫邮政列车比过去 Jesse James 时代（美国内战期间铁路巨头们为了在中西部铺设铁路而与当地人民发生了激烈的冲突）要困难得多。

而且，人们通常能够辨别出原始的纸质文档和复印件之间的区别，这点对人们来说非常重要。你不妨做一个试验，把手头的一张有效支票复制一份，星期一将支票原件拿到银行去兑现，然后星期二用复印件再试着兑现一次。你能注意到因为这种区别所以银行很容易辨别它。但是若使用电子支票，因为原始支票和复制的支票是一样，银行可能需要花相当一段时间来知道如何识别真伪。

人们通过辨别相貌、声音和笔迹来识别其他人，通过在信笺纸上签字或者盖图章等手段作为签名的证据。篡改的行为通常可以被笔迹、墨水和纸张专家检测出来。但是所有这些手段在电子方式下统统无效，因此我们需要解决问题的其他方法。

在得到各种答案之前，首先值得花点时间来考虑协议栈中网络安全应该归属何方。可能无法把它单独归结为任何单个方面。协议栈中的每一层都对网络安全做出贡献。在物理层中，可以通过将传输线封装在内含高压气体的密封管线中等措施来有效对付那些搭线窃听的攻击手段。若有人企图在管线上钻孔，气体就会泄漏，从而降低导线管中的气压，并触发报警。有些军用系统就使用这种技术。

在数据链路层，通过点对点线路上传输的数据包可以在离开机器前被加密处理，当它们进入另一台机器时再被解密。所有的细节可以都在数据链路层内部处理，上面的网络层对于数据链路层发生的事情一无所知。然而，当数据包必须要穿越多台路由器的时候，这种方法就带来了问题。因为在每台路由器上数据包必须被首先解密，然后才能做进一步的处理，这种处理方式使得数据包容易遭受来自路由器内部的攻击。而且，它也不允许只保护某些会话（比如那些涉及用信用卡在线购物的会话）而不保护其他会话。不过，这种称为链路加密（link encryption）的方法正如其名所示，它可以很容易地被应用到任何一个网络中，所以它往往还是有用的。

在网络层，可以通过安装防火墙等措施来过滤好的和坏的数据包，让好的数据包正常通过而阻止有问题的数据包进入。IP 安全（IPSec）就运行在这一层上。

在传输层，可以对端到端的整个连接进行加密，即从进程到进程的加密。为了达到最高级别的安全，端到端的安全是必需的。

最后，诸如像用户认证和不可否认性这样的问题只能在应用层上得以解决。

由于安全问题并不完全与某一层吻合，所以它也就不适合在本书前面任何一章中讨论。基于这个原因，我们用专门一章来讨论安全问题。

虽然本章篇幅比较长，技术性很强，而且也比较基础，但是，它相对来说比较独立。现有资料已完整地记载多数银行发生的安全事件。例如，松懈的安全程序、不合格的雇员以及无数的程序错误使系统易于遭受未经授权人的远程破解和所谓的社会工程师的攻击，

或诱使客户透露他们账号的详细信息。所有这些安全问题往往并不是因为聪明的罪犯搭接和窃听电话线，然后解密听到的消息而造成的。如果一个人拿着一张在马路上捡到的 ATM 卡，随便走进一家银行的分支结构并声称自己忘了 PIN 码而当场获得新的 PIN 码（凭着很好的客户关系）的话，那么世界上所有的密码系统都将无法防止这种数据窃用。从这个角度而言，Ross Anderson 的书是一本真正让人大开眼界的好书，它记录了在各个行业中曾经发生的数百起安全失败的例子，几乎所有这些例子都是由于商务管理松散或者对安全性掉以轻心造成的（这是客气的说法）（Anderson, 2001）。尽管如此，在考虑了所有的那些因素后，构建电子商务的技术基础就是加密法。

除了物理层安全以外，几乎所有的网络安全都基于密码学原理。出于这样的原因，我们将通过比较详细地研讨密码学来开始有关网络安全的学习。在 8.1 节，我们将介绍一些基本的原理；在 8.2~8.5 节中，我们将讨论一些密码学的基础算法和数据结构。然后，我们将详细地讨论如何使用这些概念来实现网络中的安全性。最后我们简短地给出有关技术和社会的进一步思考作为小结。

在我们开始之前，有必要提及哪些内容本章没有涉及。我们尽量将焦点集中在与网络有关的问题上，而不是与操作系统和应用系统有关的问题上，当然有时它们的界限难以划分。例如，本章不讨论以下内容：基于生物统计学的用户认证、口令安全、缓冲区溢出攻击、特洛伊木马、登录欺骗、诸如跨点脚本的代码注入、病毒、蠕虫和类似这些内容。所有这些问题都已经在“Modern Operating Systems”（Tanenbaum, 2007）一书的第 9 章中介绍。对系统安全方面有兴趣的读者可以参考那本书。现在让我们开始网络安全的学习之旅。

8.1 密码学

密码学（cryptography）一词来自于希腊语中的短语“secret writing”（秘密书写）。它有着辉煌而又悠久的几千年历史。在这一节中我们只是勾画出其中一些亮点，然后介绍一些后面学习所需的背景知识。有关完整的密码学历史，你可以阅读（Kahn, 1995）。要想全面地了解安全和密码算法的最新现状、协议和应用，以及有关材料，请参考（Kaufman 等, 2002）。想了解更数学化的方法，请参考（Stinson, 2002）；对于那些不那么数学化的方法，可以参考（Burnett 和 Paine, 2001）。

专业人员对密码和编码作了明确的区分。密码（cipher）是指逐个字符或者逐个位地进行变换，它不涉及信息的语言结构。相反，编码（code）则是指用一个词或符号来代替另一个词。尽管编码学有着非常光荣的历史，但现在已经很少被使用了。在历史上设计最为成功的编码系统是二次世界大战中美国军队在太平洋战场上使用的编码。他们的做法很简单，让纳瓦霍印第安人用专门的纳瓦霍语言来交流军事术语，例如，用 chay dagahi nail tsaidi（字面意思是“慢行者之克星”）来表示反坦克武器。纳瓦霍语言是一门以音调为主的语言，非常复杂，而且没有书面的形式。日本方面根本没有一个人懂这门语言。

在 1945 年的 9 月，圣地亚哥联合会如此描述美国的编码系统：“三年来，无论什么时候海军登陆时，日本人总是听到一连串奇怪的汩汩声，同时夹杂着像是和尚念经的声音和

热水瓶倒空时发出的声音。”日本人始终未能破解这套编码系统，纳瓦霍编码的许多配音者由于突出的表现和勇敢精神而获得了很高的军事荣誉。美国破解了日本的编码，而日本却未能破解纳瓦霍编码，这个事实本身为美国在太平洋战争中赢得胜利起到了至关重要的作用。

8.1.1 密码学概论

历史上，有4类人用到了密码学，并且为之作出了贡献，他们是军事人员、外交人员、写日记者和情侣。在这4组人中，军事人员扮演了重要的角色，而且几个世纪以来他们不断完善着这个领域。在军事组织内部，需要加密的消息通常被交给收入低微的低级译码员来加密和传输。庞大的消息数量不能仰仗少数精英专家来完成。

在计算机出现以前，密码学的一个主要限制在于译码员执行所需变换的能力，尤其在战场上往往缺乏相应的装备。另一个限制是很难从一种密码方法切换到另一种密码方法，因为这需要重新训练一大批人。然而，由于译码员有可能被敌人俘虏，所以，在必要的时候能及时地更换密码方法变得非常关键。这些相互矛盾的需求导致产生了如图8-2所示的模型。

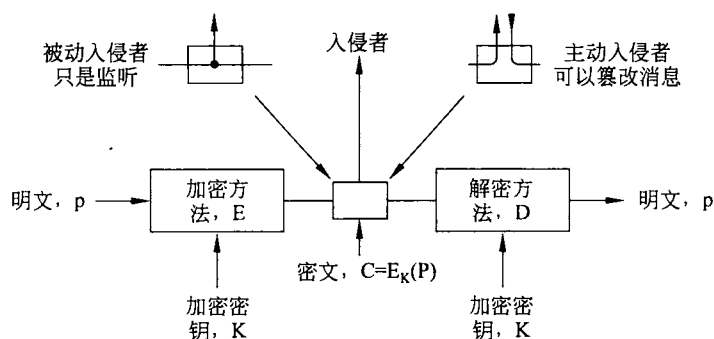


图 8-2 加密模型（对称密钥密码）

待加密的消息称为明文（plaintext），它经过一个以密钥（key）为参数的函数变换，这个过程的结果就是所谓的密文（ciphertext），然后通常由无线电或者通信员传送出去。我们假设敌人或者入侵者（intruder）可以听到和精确地复制整个密文。然而，与目标接收者不同的是，他不知道解密所用的密钥是什么，所以他们无法很轻易地对密文进行解密。有时候入侵者不仅可以监听通信信道（被动入侵者），而且可以将消息记录下来并且在以后某个时候回放出来，或者插入他自己的消息，或者在合法消息到达接收方之前篡改消息内容（主动入侵者）。破译密码的技术就是所谓的密码分析学（cryptanalysis），而它和设计密码的密码编码学（cryptography）合起来统称为密码学（cryptology）。

用一种合适的表示法将明文、密文和密钥的关系体现出来，往往会非常有用。我们将使用 $C = E_K(P)$ 表示用密钥 K 加密明文 P 得到密文 C 。类似地， $P = D_K(C)$ 表示解密密文 C 得到明文 P 。由此可以得到：

$$D_K(E_K(P)) = P$$

这种表示法说明 E 和 D 只是数学函数，事实上它们也确实是这样。唯一的诀窍是它们

都是带两个参数的函数，并且我们将其中一个参数（密钥）写成下标的形式而不写成实参的形式，从而将它与消息本身区别开来。

密码学的基本规则是假定密码分析者一定知道加密和解密所使用的方法。换句话说，密码分析者知道图 8-2 中加密方法 E 和解密方法 D 的所有细节。当老方法被泄漏或者认为它们已被泄漏后，就需要花费大量的精力来发明、测试和安装新的算法，因此将加密算法本身保持秘密的做法是不现实的。当一个算法已不再保密时而仍然认为它是保密的，这将会带来更大的危害。

这正是密钥的用武之处。密钥由一段相对比较短的字符串构成，它相当于从大量潜在的加密方法中选择了一种加密方法。一般的加密方法可能几年才会发生变化，与密钥可以根据需要频繁地被改变。因此，我们的基本模型是一个稳定的、广泛公开的通用方法，它用一个秘密的、易改变的密钥作为参数。“让密码分析者知道加解密算法，并且把所有的秘密信息全部放在密钥中”这种思想称为 **Kerckhoff 原理** (Kerckhoff's principle)，这是用佛兰德军事密码学家 Auguste Kerckhoff 的名字来命名的，他在 1883 年第一次提出了这种想法 (Kerckhoff, 1883)。因此，我们有

Kerckhoff 原则：所有的算法必须是公开的而密钥是保密的。

算法的不保密性怎么强调都不过分。企图使算法保持秘密的做法称为**安全模糊**，这种做法永远不会奏效。而且，将算法公开，让密码设计者可以自由地与大量学院派的密码学家进行交流探讨，这些密码学家总是期望能破解密码系统，从而可以发表论文，来证明自己有多聪明。如果一个密码算法被公开 5 年以后，尽管在此期间许多专家试图破解该算法，但是无人能够成功，那么，这个算法应该是非常可靠的。

由于真正的秘密在密钥中，所以它的长度是一个非常重要的设计要素。请考虑一个简单的号码锁，它通常的原理是按照顺序输入数字，这点大家都知道，但是锁的号码（即密钥）却是保密的。如果密钥长度是两个数字，则意味着共有 100 种可能。若密钥长度为三个数字，则意味着共有 1000 种可能；同样地，六位数字的密钥意味着 100 万种可能。密钥越长，则密码分析者要消耗的工作因子 (work factor) 就越高。通过穷举搜索整个密钥空间来破解密码系统的工作因子随着密钥长度增加而呈指数递增。保密性来自于强而牢固的（但是公开的）算法和密钥的长度。为了防止你的孩子阅读你的电子邮件，64 位密钥足够了。对于常规的商业用途，至少应该使用 128 位密钥。对于大规模的政府机构，则至少需要 256 位的密钥，而且需要的话应该越长越好。

从密码分析者的角度来看，密码分析问题有 3 个主要的变种。当他得到了一定量的密文，但是没有对应的明文时，他面对的是**唯密文** (ciphertext only) 问题。报纸上猜谜栏目中的密码难题就属于这一类问题。当密码分析者有了一些相匹配的密文和明文时，密码分析问题称为**已知明文** (known plaintext) 问题。最后，当密码分析者能够加密某一些他自己选择的明文时，问题就变成了**选择明文** (chosen plaintext)。如果可以问密码分析者诸如“ABCDEFGHJKLM 经过加密之后是什么？”之类的问题，那么破解报纸上的那种密码问题就容易了。

密码编码学领域中的新手通常作这样的假设：如果一个密码能够承受密文攻击，那么它就是安全的。这个假设非常幼稚。在许多情况下，密码分析者可以正确地猜测出明文的某些部分。例如，当你向其他机器发出请求时，许多计算机做的第一件事情是回复“login:”。

一旦有了某些相匹配“明文-密文”后,密码分析的工作就容易得多。为了保证安全起见,密码设计者应该保守一点,并且确信即使对手能够加密任意数量的选择明文,密码系统也不会被攻破。

在历史上,加密方法被分成两大类:置换密码和替代密码。现在我们简要地介绍这两种密码,正好作为现代密码学的背景信息。

8.1.2 置换密码

在置换密码(substitution cipher)中,每个字母或者每一组字母被另一个字母或另一组字母取代,从而将原来的字母掩盖起来。最古老的密码之一是凯撒密码(Caesar cipher),它因Julius Caesar而得名。在这种方法中,a变成D,b变成E,c变成F,...,z变成C。例如,attack变成DWWDFN。在例子中,明文以小写字母给出,密文则使用大写字母。

凯撒密码的一种略微一般化方案是,允许明文按字母顺序被移动k个字母,而并不总是移动3个位置。在这种情况下,k变成了这种循环移动字母表的通用加密方法的一个密钥。凯撒密码也许确实欺骗了庞培(Pompey),但自那以后再也没有骗过别人。

接下来的改进是,让明文中的每个符号(为了简化起见,这里假设为26个字母)映射到其他某一个字母上,例如,

明文: a b c d e f g h i j k l m n o p q r s t u v w x y z

密文: Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

采用这种“符号对符号”置换策略的通用系统称为单字母置换(monoalphabetic substitution)密码,其密钥是对应于整个字母表的26字母串。对于上面的密钥,明文attack被变换为密文QZZQEA。

初看起来,这似乎是一个非常安全的系统,因为虽然密码分析者了解通用的系统(即字母对字母的置换),但是它不知道到底使用了哪一个密钥,而密钥的可能性共有 $26! \approx 4 \times 10^{26}$ 种。与恺撒密码不同的是,要试遍所有这么多可能的密钥并不是一种可行的做法。即使得到一种方法只需要花费1纳秒的时间,但用百万次计算机芯片进行并行计算也需要花费一万年的时间才能试遍所有可能的密钥。

不过,事实上只要知道相对少量的密文,密码就很容易被破解。基本的攻击手段利用了自然语言的统计特性。例如,在英语中,e是最常见的字母,其次是t、o、a、n、i等。最常见的两字母组合(或者两字母连字)是th、in、er和an。最常见的三字母组合(或者三字母连字)是the、ing、and和ion。

密码分析者为了破解单字母密码,他首先计算所有字母在密文中出现的相对频率,然后他也许会试探将最常见的字母分配给e,次常见的字母分配给t。接下来他将查看三字母连字,找到比较常见的形如tXe的三字母组合,这强烈地暗示着其中的X是h。类似地,如果模式thYt出现得很频繁的话,则Y可能代表了a。有了这些信息以后,他可以查找频繁出现的形如aZW的三字母组合,它很有可能是and。通过猜测常见的字母、两字母连字和三字母连字,并且利用元音和辅音的各种可能组合,密码分析者就可以逐个字母地构造出试探性的明文。

另一种做法是猜测一个可能的单词或者短语,例如,考虑以下这段来自于一家会计事

务所的密文（分成5个字符为一组）：

CTBMN BYCTC BTJDS QXBNS GSTJC BTSWX CTQTZ CQVUJ
QJSGS TJQZZ MNQJS VLNSX VSZJU JDSTS JQUUS JUBXJ
DSKSU JSNTK BGAQJ ZBGYQ TLCTZ BNYBN QJSW

在会计事务所的消息中，一个可能的单词是 financial（财经）。我们知道在 financial 这个单词中有一个重复的字母（i），并且这两个 i 之间有 4 个其他的字母，根据这样的知识，我们在密文中查找相隔 4 个位置的重复字母。我们可以找到 12 个地方，分别在 6、15、27、31、42、48、56、66、70、71、76 和 82 位置上。然而，只有其中两个地方，即 31 和 42，它的下一个字母（对应于明文中的 n）重复出现在正确的位置上。而在这两者之中，只有 31 有正确的 a 位置（考虑在 financial 中有两个 a），所以，我们知道 financial 从位置 30 开始。以此为出发点，利用英语文本的频率统计规律可以很容易地推断出密钥，从而能找出最终的全部单词。

8.1.3 替代密码

置换密码保留了明文符号的顺序，但是将明文伪装起来了。与此相反，替代密码（transposition cipher）重新对字母进行排序，但是并不伪装明文。图 8-3 给出了一个常见的替代密码：列换位。该方案用一个不包含任何重复字母的单词或者短语作为密钥。在这个例子中，密钥是 MEGABUCK。密钥的用途是对列顺序进行排列，第 1 列是密钥字母中最靠近字母表起始位置的那个字母下面那列，以此类推。明文按水平方向的行来书写，如果有必要的话填满整个矩阵。密文则被按列读出，从密钥字母最小的那列开始。

M	E	G	A	B	U	C	K	
7	4	5	1	2	8	3	6	
p	l	e	a	s	e	t	r	明文：
a	n	s	f	e	r	o	n	pleasetransferonemilliondollarsto
e	m	i	l	l	i	o	n	myswissbankaccountsixtwo
d	o	l	l	a	r	s	t	
o	m	y	s	w	i	s	s	密文：
b	a	n	k	a	c	c	o	AFLLSKSOSELAWAIATOOSSCTCLNMOMANT
u	n	t	s	i	x	t	w	ESILYNTWRNNTSOWDPAEDOBUEOERIRCXB
o	t	w	o	a	b	c	d	

图 8-3 替代密码

为了破解替代密码，密码分析者首先要明白，自己是在破解一个替代密码。通过查看 E、T、A、O、I、N 等字母的频率，很容易就可以看出它们是否吻合明文的常规模式。如果是的话，则很显然这是一种替代密码，因为在这样的密码中，每个字母代表的是自己，从而保持了字母出现的频率分布。

接下来要猜测共有多少列。在许多情况下，从环境信息中或许可以猜到一个可能的单词或者短语。例如，假定我们的密码分析者怀疑消息中的某个地方出现了明文短语 milliondollars。他观察到在密文中出现的两字母组合 MO、IL、LL、LA、IR 和 OS 是因为这个短语回绕的结果。密文字母 O 跟在密文字母 M 的后面（即在第 4 列的垂直方向上它们是相邻的），这可能是因为它们在短语中被一段等于密钥长度的距离所隔开。如果密钥长度为 7 的话，则两字母组合 MD、IO、LL、LL、IA、OR 和 NS 就会出现。实际上，对于

每一个密钥长度，在密文中都会出现一组不相同的两字母组合。通过检查不同的可能性，密码分析者往往很容易就能够确定密钥的长度。

最后的步骤是给列顺序。当列数比较小，假设有 k 列时，依次检查 $k(k-1)$ 个列队中的每一个它的出现频率是否和英文明文相同两字母组出现的频率相同。假定频率最接近的那一对的放置位置是正确的，我们可以尝试让剩下的每一列跟在这一对的后面，两字母组合和三字母组出现频率最匹配的那列假设是正确的。用相同的方法继续找到下一列。整个过程继续下去，直至找出潜在的列顺序。到这时候，明文可以确定是否破解成功了（比如，如果出现 *miloin* 的话，很明显就知道错误在哪里了）。

有些替代密码接受一个固定长度的块作为输入，并产生一个固定长度的块作为输出。通过给出一个指明字符输出顺序的列表就可以完整地描述这样的密码。例如，图 8-3 中的密码可以被看作一个 64 字符块的密码。它的输出是 4、12、20、28、36、44、52、60、5、13、…，62。换句话说，第 4 个输入字符 *a* 首先被输出，然后是第 12 个字符 *f*，以此类推。

8.1.4 一次性密钥

要想构建一个不可被攻破的密码其实是非常容易的，相应的技术在几十年前就已经知道。首先选择一个随机位串作为密钥，然后将明文转变成一个位串（比如使用明文的 ASCII 表示法）。最后，逐位计算这两个串的异或（XOR）值，这样得到的密文是不可能被破解的。这是因为在数量足够大的密文样本中，每个字母的出现频率是相等的，两字母组合，三字母组合等的出现频率也都是如此。这种方法称为一次性密钥（one-time pad），它可以抵挡现在和将来的攻击，无论入侵者的计算能力有多么强大都是如此。理由源于信息理论：因为任何给定长度的明文都有相同的可能性，所以消息中根本不存在任何可以用来解密的信息。

图 8-4 给出了一个一次性密钥用法的例子。首先，消息 1 “I love you.” 被转换成 7 位 ASCII 码。然后选择一个一次性密钥 Pad1，并且与消息 1 进行异或（XOR）操作得到密文。密码分析者可以试验所有可能的一次密钥，并检查每个密钥所对应的明文。例如，图中列出的一次性密钥 Pad2 可以被用来做试验，结果得到明文 2 “Elvis lives”，这个结果有点似是而非（关于这方面的讨论超出了本书的范围）。事实上，对于每一个长度为 11 个字符的 ASCII 明文，就有一个生成此明文的一次性密钥。这也正是我们所说的在密文中没有任何信息的原因：你总是可以得到任何一条正确长度的消息。

消息 1:	1001001 0100000 1101100 1101111 1110110 1100101 0100000 1111001 1101111 1110101 0101110
一次性密钥 1:	1010010 1001011 1110010 1010101 1010010 1100011 0001011 0101010 1010111 1100110 0101011
密文:	0011011 1101011 0011110 0111010 0100100 0000110 0101011 1010011 0111000 0010011 0000101
一次性密钥 2:	1011110 0000111 1101000 1010011 1010111 0100110 1000111 0111010 1001110 1110110 1110110
明文:	1000101 1101100 1110110 1101001 1110011 0100000 1101100 1101001 1110110 1100101 1110011

图 8-4 利用一次性密钥进行加密，通过其他一次性密钥可从密文获得任何可能明文

在理论上一一次性密钥的确很好，但在实际使用中却有若干缺点。首先，一次性密钥无法记忆，所以发送方和接收方必须随身携带书面的密钥副本。如果任何一方被敌人捕获，则显然书面的密钥是一个很大的威胁。除此之外，可被传送的数据总量受到可用密钥数量

的限制。如果一名间谍非常走运，发现了一批极有价值的数据，他可能由于密钥已被用尽而无法将这批数据传回总部。另一个问题是，这种方法对于丢失字符或者插入字符非常敏感。如果发送方和接收方失去了同步，那么失去同步后面的所有数据都变成了垃圾。

随着计算机的出现，一次性密钥方法对于某些应用可能会变得具有实用价值。密钥源可以是一片特殊的 DVD，它包含了几千兆字节的信息，如果它被放在 DVD 电影箱中运输，并且开头部分加上几分钟电影片段，那么这样的 DVD 甚至不会招人怀疑。当然，在千兆位速率的网络上，每隔 30 秒钟就必须插入一片新的 DVD 可能非常烦人。由于在发送消息之前，必须通过人将 DVD 从发送方带给接收方，这极大地降低了一次性密钥方法的实际使用性。

量子密码

有趣的是，针对如何在网络上传输一次性密钥问题，可能真的存在一种解决方案。这种方案来源于一种看来不太可能的源：量子机。这个领域现在仍然属于试验性质，但是初始的试验非常成功。如果它能做得完美并且很有效，那么，所有的密码系统都可以利用一次性密钥方法来完成，因为已经证明一次一密的方法是绝对安全的。下面我们简短地介绍这种量子密码学（quantum cryptography）的工作原理，尤其是介绍一个称为 BB84 的协议，该协议的名字来源于其作者的名字和发表年份（Bennet 和 Brassard, 1984）。

假设有一个用户 Alice 希望与第二个用户 Bob 建立一个一次性密钥。这里 Alice 和 Bob 称为主角（principal），他们是我们故事中的主角。例如，Bob 是一个银行家，Alice 希望与他做一些商务交易。自从 Ron Rivest 在很多年前引入这两个名字后，几乎所有关于密码学的论文和书籍在叙述的时候都使用 Alice 和 Bob 作为虚拟主角。密码学家都喜欢传统，所以，如果我们使用 Andy 或者 Barbara 作为主角的话，就没有人会相信这一章的内容了。所以，我们还是沿用传统的做法。

如果 Alice 和 Bob 能够建立一个一次性密钥，那么，他们就可以用该密钥来安全地通信了。问题是：如果不像前面提到的那样交换 DVD 的话他们该如何建立一次性密钥呢？我们可以假定 Alice 和 Bob 在一根光纤的两端，通过这根光纤他们可以发送和接收光脉冲。然而，一个超级强大的名叫 Trudy 入侵者能够割断光纤并插入一个接头，这样 Trudy 可以读取两个方向上的所有数据，也可以往两个方向上发送假的消息。这种情形对于 Alice 和 Bob 来说，似乎已经没有希望进行安全通信了，但是，量子密码学却为他们带来了一线新的曙光。

量子密码学基于这样一个事实：光是以一种极小的称为光子（photon）的包的形式传递的，并且光子具有某些偏振特性。而且，光在通过一个偏振滤光镜时可以被调整到一个方向上，戴太阳镜的人或者摄影师都知道这个事。如果将一束光（即一个光子流）通过一个偏振滤光镜，则该光束中的所有光子都将被偏到滤光镜的轴向（比如垂直方向）上。如果现在光束再通过第二个偏振滤光镜，则从第二个滤光镜出来的光的强度将与两轴之间夹角的余弦平方成正比。如果这两个轴相互垂直的话，那么所有的光子都通不过。两个滤光镜的绝对方向并不重要，关键是它们之间的夹角。

为了产生一次性密钥，Alice 需要两组偏振滤光镜，第一组偏振滤光镜由一个垂直滤光镜和水平滤光镜组成，这种选择称为直线基（rectilinear basis）。这里的一个基只是一个坐

标系统。第二组偏振滤光镜一样，但是旋转了 45°，所以，一个偏振滤光镜的方向是从左下至右上，另一个偏振滤光镜的方向是从左上至右下。这种选择称为**对角基**（diagonal basis）。因此，Alice 有两个基，她可以根据需要快速地将这些基插入到她的光束中。实现时，Alice 并没有 4 个独立的偏振滤光镜，而是一个晶体，该晶体的偏振方向可以通过电子方式以极快的速度切换到 4 个允许方向中的任何一个。Bob 也有一套与 Alice 相同的设备。Alice 和 Bob 两个人都有两组可用的基，这对于量子密码系统是非常关键的。

对于每一组基，Alice 现在将一个方向作为 0，另一个方向作为 1。在下面介绍的例子中，我们假设她选择垂直方向为 0，水平方向为 1。另外，她也选择从左下至右上方向为 0，从左上至右下方向为 1。她通过明文方式将这些选择发送给 Bob。

现在 Alice 选择一个**一次性密钥**，比如说她利用一个随机数发生器（这本身就是一个非常复杂的主题）来生成该密钥。然后她逐位地将密钥传送给 Bob，在传送每一位的时候她随机地选择其中一个基。为了发送每一位，她的光子枪发射出来的光子已经被正确地偏振到她为这一位所选择使用的基上。例如，她可能选择对角基、直线基、直线基、对角基、直线基等。为了用这些基来发送她的一次性密钥 1001110010100110，她将会发送图 8-5(a) 中所示的光子。给定了一次性密钥和基的序列之后，每一位的偏振方向也被唯一地确定下来。像这样一次发送一个光子的数据位称为**量子位**（qubit）。

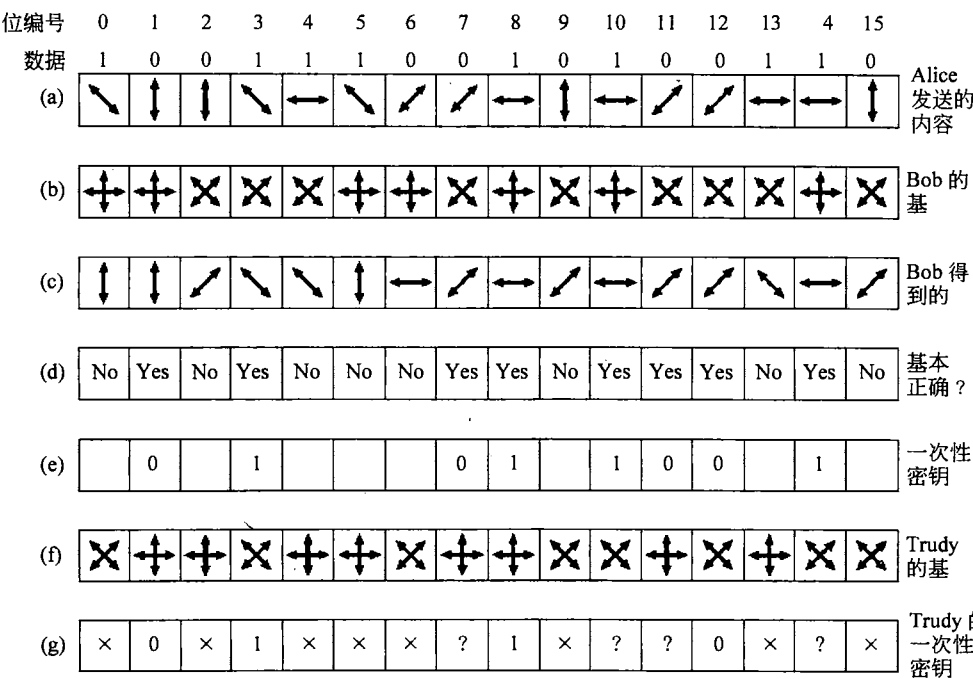


图 8-5 一个量子密码系统示例

Bob 并不知道 Alice 使用了哪些基，所以他随机地为每一个到来的光子选择一个基，并使用这个基，如图 8-5(b)所示。如果他选择了正确的基，则他会得到正确的数据位。如果选择的基不正确，则得到一个随机的位，因为如果一个光子被发射到一个与它自己的偏振方向成 45°角的滤光器上，那么它将会随机地跳到滤光镜的偏振方向上，或者跳到与滤光镜方向垂直的偏振方向上，这两者的概率是相等的。光子的这种性质正是量子机的基础。

因此,有些位是正确的,而有些位是随机的,但是 Bob 并不知道哪些是正确的、哪些是不正确的。Bob 得到的结果如图 8-5(c)所示。

那么, Bob 如何知道他选择的哪些基是正确的、哪些基是不正确的呢?他简单地告诉 Alice,他为明文中的每一位使用了哪个基;然后 Alice 告诉他,明文中哪些是正确的,哪些是不正确的,如图 8-5(d)所示。利用这些信息,双方就可以根据正确的猜测结果获得一个位串,如图 8-5(e)所示。平均来说,这个位串的长度将是原始位串的一半,但是由于双方都知道这个位串了,所以他们可以将该位串用作一次性密钥。Alice 所要做的事情仅仅是传输一个略微超过所需长度之两倍的位串,于是她和 Bob 就有了一个所需长度的一次性密钥。问题解决了!

但请稍等。我们忘了还有 Trudy 呢。假定她对于 Alice 所说的话非常好奇,所以她切断光纤并插入了她自己的检测器和发射器。对她来说不幸的是她并不知道每个光子使用了哪一个基。她所能够做的就是像 Bob 一样,随机地为每个光子选择一个基。图 8-5(f)显示了她所选择的一个例子。当 Bob 后来用明文向 Alice 报告他使用了哪些基,并且 Alice 也用明文告诉他哪些基是正确的時候, Trudy 现在也知道她得到的哪些位是正确的,哪些位是错误的。在图 8-5 中,她知道以下位是正确的:0、1、2、3、4、6、8、12 和 13。但是,从图 8-5(d)的 Alice 应答中,她知道只有 1、3、7、8、10、11、12 和 14 位才是一次性密钥的组成部分。她猜中了其中的 4 位(即 1、3、8 和 12 位),并且也捕获到了正确的位信息。其他的 4 位(7、10、11 和 14 位),她猜测错误,所以她不知道真正传输的位是什么。因此, Bob 从图 8-5(e)中得知,一次性密钥的前 8 位为 01011001,但是, Trudy 仅仅得到了 01?1??0?,如图 8-5(g)所示。

当然, Alice 和 Bob 知道 Trudy 可能已经捕获了他们的一次性密钥的一部分,所以他们希望进一步减少 Trudy 拥有的信息。他们只要对这个一次性密钥做一个变换就可以做到这一点。例如,他们可以将一次性密钥分成 1024 位的块,然后对每一块做平方操作,从而构成 2048 位数值,再将这些 2048 位数拼接起来当作一次性密钥。Trudy 只有一部分位串信息,所以她无法产生位串的平方,因此她什么也得不到。从原始的一次性密钥变换到另一个削弱了 Trudy 所知信息,这个变换称为保密增强(privacy amplification)。实际上,往往采用这种输出位依赖于所有输入位的复杂变换方法,而不用简单的平方。

可怜的 Trudy。她不仅对一次性密钥一无所知,而且她的存在也不再是一个秘密了。毕竟,她必须把接收到的每一位转发给 Bob 才能欺骗他,让他以为自己是在跟 Alice 通话。问题在于她 Trudy 能做的最好事情也只是用她接收用的偏振来传输她收到的每一量子位,而其中大约有一半是错误的,从而导致 Bob 的一次性密钥中有很多错误。

当 Alice 最后开始发送数据时,她使用一种重量级的前向纠错码对数据进行编码。从 Bob 的角度来看,一次性密钥中的 1 位错误与 1 位传输错误是相同的,他总是收到错误的位。如果有足够的前向纠错能力,那么不管任何错误他都能够恢复出原始的消息,同时他可以很容易地计算出有多少错误得到了纠正。如果这个数值大大超过了设备的期望错误率,他就能肯定 Trudy 已经搭接在线路上了,于是可以根据情况做出响应(比如告诉 Alice 切换到一个无线电信道上,或者打电话给警察局,等等)。如果 Trudy 有办法复制光子,从而她可以监视其中一个光子,并且将另一个同样的光子发送给 Bob,那么,她就可以将自己隐藏起来,避免被 Bob 发现。但是,迄今为止人们尚未发现有理想的方法可以复制光子。不

过，即使有一天 Trudy 能够复制光子，量子密码学用于建立一次性密钥的价值也不会被减弱。

尽管研究人员已经可以在超过 60 千米距离的光纤上运行量子密码系统，但是装备非常复杂，而且极其昂贵。不过，量子密码学的思想仍然很有前途。有关量子密码学的更多信息，请参考（Mullins，2002）。

8.1.5 两个基本的密码学原则

在本章我们将学习许多不同的密码系统，首先要了解这些密码系统背后的两条基本原则，这对于理解后面的内容非常重要。注意，违反它们就是在冒险。

冗余度

第一条原则：所有被加密的消息必须包含一定的冗余信息，也就是说，这些信息对于理解这条消息来说是不必要的。通过一个例子或许能看清楚为什么需要冗余信息。请考虑一家名为 The Couch Potato (TCP) 的邮购公司，它有 60 000 多种产品。想象该公司的效率很高，TCP 公司的程序员决定，每条订购消息应该包含一个 16 字节的顾客名以及一个 3 字节的数据字段（一个字节用于购买数量，另 2 个字节用于产品编号）。最后 3 个字节用一个非常长的密钥进行加密，并且只有顾客和 TCP 公司才知道该密钥。

初看起来这种方案似乎非常安全，从某种意义上来看它确实是安全的，因为被动的入侵者不可能解密消息。不幸的是，它有一个致命的缺陷使得它毫无用处。假定有一个最近刚刚被解雇的员工想要报复 TCP 公司。在离开之前，她带走了顾客的名单。她连夜编写了一个程序来产生伪造的订单，这些订单的客户名是真实的。由于她没有拿到客户们的密钥，所以她在最后的 3 个字节中只是放了一些随机数，然后给 TCP 公司发送了几百份订单。

当这些订购消息到达公司时，TCP 公司计算机利用客户的名字找到对应的密钥，并解密消息中的最后 3 个字节。对于 TCP 公司来说，不幸的是几乎所有的 3 字节消息都是有效的，所以，计算机开始打印发货指令。虽然客户订购 837 套儿童秋千，或者 540 个沙箱，这看起来非常奇怪，但计算机知道的就是这些信息，也许顾客可能正在筹建一个获准特许经营的游乐场。以这种方式，一个主动的入侵者（离职员工）即使并不理解她自己的计算机所产生的消息，她也可以通过这种方式造成大量的麻烦。

实际上，通过在所有的消息中增加一定的冗余度就可以解决这个问题。例如，如果订购消息被扩展到 12 个字节，前 9 个字节必须为 0，那么，这种攻击便不再奏效，因为离职员工不可能产生大量的有效消息。从这个例子得到的启示是：所有的消息必须包含足够大的冗余度，因而主动入侵者发送的随机垃圾消息不可能有机会被解释为有效的消息。

然而，增加冗余度也使得密码分析者更加容易破解消息。假设邮购业务的竞争非常激烈，TCP 公司的主要竞争对手 Sofa Tuber 极力想知道 TCP 公司卖出了多少个沙箱。因此，他们搭接在 TCP 公司的电话线上。在原来的 3 字节消息的方案中，密码分析手段几乎不可能成功，因为在试验了一个密钥之后，分析者无从判断他的猜测是否正确。毕竟，从技术上讲，几乎每一条消息都是合法的。但是，在采用了新的 12 字节的方案以后，分析者可以很容易地区分出有效的消息和无效的消息。因此，我们得到了：

密码学原则 1: 消息必须包含一定的冗余度

换句话说, 在解密了一条消息以后, 接收者必须能够通过简单的检查手段和通过一个简单的计算来判断该消息是否有效。这种冗余是必要的, 它可以用来阻止主动入侵者发送垃圾信息, 或者防治接收者被诱骗解密垃圾信息, 然后处理所谓的“明文”。然而, 同样这冗余信息也使得被动入侵者更加容易破解系统, 所以这里有点矛盾。而且, 冗余信息不应该采用在消息的开头或者末尾加上 n 个 0 的形式, 因为在某些密码算法上加解密这样的消息可以得到一些更有预测性的结果, 从而使密码分析者的工作更加容易。CRC 多项式编码比加 0 的做法好得多, 因为接收者很容易验证消息, 而密码分析者却需要更大的工作量。更好的做法是使用密码散列, 这个概念我们将在本章后面学习。此刻, 我们就认为它是一种更好的 CRC。

再回到稍早前的量子密码上, 我们可以看到冗余原则在那里扮演了多么重要的角色。由于 Trudy 截获了光子, 所以造成 Bob 的一次性密钥中有些位是错误的。Bob 需要入境消息具备一定的冗余度来判断该消息是否出现了错误。一种非常原始的冗余形式是将消息重复两遍。如果两份副本不相同, 那么 Bob 就知道要么光纤上噪声很大, 要么有人篡改了传输信息。当然, 将所有信息都发送两份这种做法有点矫枉过正, 采用海明码或者里德所罗门编码可以更加有效地实行检错和纠错。但是, 应该明确引入冗余度能区分有效消息和无效消息, 特别是在面对主动入侵者的情况下。

新鲜度

第二条密码学原则是, 必须采取某些措施确保每条接收到的消息可被验证是新鲜的, 也就是说, 它是最近刚发送出来的。为了预防主动入侵者重放老消息, 这种措施是必要的。如果没有相应的措施, 那么, 我们的例子中的离职员工就可以搭接在 TCP 公司电话线上, 然后不停地重复以前发送的有效消息。因此:

密码学原则 2: 需要采取某种方法来对抗重放攻击

一种措施是在每条消息中包含一个仅仅在一段时间内 (比如 10 秒钟) 有效的时间戳。然后, 接收方只要保留该消息 10 秒钟, 并且将新到达的消息与以前的消息进行比较就可以过滤掉重复的消息。10 秒钟之前到达的消息可以被丢弃, 因为凡是在 10 秒钟以后发送的任何重放消息将被当作太老的消息而遭到拒绝。除了时间戳以外的其他措施将在本章后面讨论。

8.2 对称密钥算法

现代密码学使用了与传统密码学相同的思想 (替代和置换), 但是它的重点有所不同。传统上, 密码设计者使用了非常简单的算法。现在, 情形完全相反: 密码设计的目标是使加密算法尽可能地错综复杂, 因而即使密码分析者获得了大量的选择密文, 在没有密钥的情况下他也不能够推测出明文。

我们在本章中将要学习的第一类加密算法称为对称密钥算法 (symmetric key algorithm), 因为它们使用同样的密钥来完成加密和解密操作。图 8-2 显示了对称密钥算法的用法。尤其是, 我们将焦点集中在块密码 (block cipher), 它接受一个 n 位的明文块作为

输入，并利用密钥将它变换成 n 位的密文块。

密码算法既可以用硬件来实现（为了速度），也可以用软件来实现（为了灵活）。虽然在本书中的主要注意力集中在算法和协议上，而算法和协议本身是独立于它们的具体实现的，但简要提及有关构造密码系统的硬件或许饶有趣味。替代和转置密码可以用简单的电子电路来实现。图 8-6(a)显示了一种称为 P 盒（P-box）的设备（这里的 P 代表数学中的排列（permutation）），P 盒的效果相当于对一个 8 位输入实现一个替代操作。如果这 8 位从上到下被指定为 01234567，则这个特定 P 盒的输出就是 36071245。通过适当的内部连线方式，P 盒可以执行任何一个替代操作，而且在实践中可以达到光的速度，因为这中间没有涉及任何计算过程，只是信号传播而已。这种设计符合 Kerckhoff 原则：攻击者知道对输入的位重新排序的通用方法，但他不知道的哪些位应该出现在哪里。

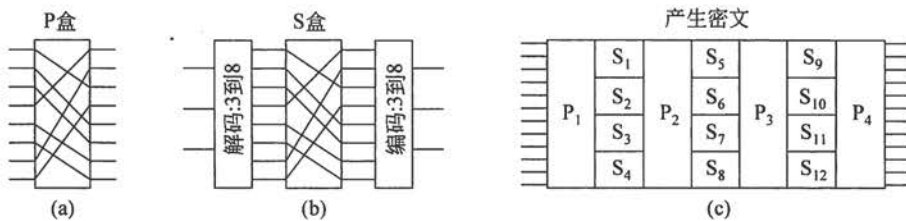


图 8-6 乘积密码的基本元素
(a) P 盒；(b) S 盒；(c) 乘积

置换由 S 盒（S-box）完成，如图 8-6(b)所示。在这个例子中，输入是 3 位明文，输出是 3 位密文。首先，3 位输入从第一阶段的 8 根输出线中选择其中一根线，并将它设置为 1，所有其他的线均为 0。第二阶段是一个 P 盒。第三阶段再将选中的输入线编码成二进制。按照图中显示的连线方式，如果依次输入 8 个八进制数 01234567，则输出序列将是 24506713。换句话说，0 被 2 代替，1 被 4 代替，等等。同样地，通过正确地设置 S 盒内部的 P 盒，任何一种置换都可以用 S 盒来完成。而且，这样的设备可以直接用硬件来构造，从而获得极快的速度，因为编码器和解码器只有一个或者两个亚纳秒（subnanosecond）门延迟，而通过 P 盒的传播延迟可能小于 1 微微秒（picosec）。

只有当我们将整个系列的盒叠加起来构成乘积密码（product cipher）时，这些基本元素的真正威力才显示出来，如图 8-6(c)所示。在这个例子中，第一步（ P_1 ）将 12 根输入线做替代操作（即重新排列）。第二步将输入分成 4 个 3 位一组，每一组被独立于其他（ $S_1 \sim S_4$ ）做置换操作。这样的安排展示了一种通过多个小 S 盒来近似达到一个大 S 盒效果的方法。因为小盒子在硬件实现上更现实（例如，一个 8 位 S 盒可以实现一个 256 项的查询表）而大 S 盒笨重难以制造（例如一个 12 位的 S 盒其中间步骤将至少需要 $2^{12}=4096$ 根交叉线）。虽然这种方法缺乏通用性，但是它仍然非常有用。通过在乘积密码中加入充分大量的处理步骤，获得的输出可以是输入的极其复杂函数。

对 k 位输入执行一个乘积密码算法以产生一个 k 位输出，这非常常见。典型情况下， k 介于 64~256。在硬件实现中，通常至少有 10 个物理步骤而不像图 8-6(c)那样只有 7 个步骤。而在软件实现中，乘积密码可以被编写成一个至少执行 8 次迭代的循环，每次迭代都在 64~256 位数据块的子块上执行类似 S 盒的置换操作，然后通过一个替代操作将这些 S 盒的输出全部混合起来。通常在起始处有一个特殊的初始替代操作，在末尾也有一个替代

操作。在许多文献中，这样的迭代称为轮 (round)。

8.2.1 DES——数据加密标准

1977年1月，美国政府采纳了IBM开发的一个乘积密码作为非机密信息的官方标准。这个密码算法，即数据加密标准 (DES, Data Encryption Standard) 已被工业界广泛应用于安全产品中。它最初的形式已经不再安全了，但是它的一个修订形式仍然非常有用。现在我们来学习 DES 的工作原理。

DES 的基本结构如图 8-7(a)所示。明文按 64 位数据块被加密，生成 64 位密文。DES 算法以一个 56 位密钥作为参数，它共有 19 个步骤。第一步是一个与密钥无关的替代操作，它直接作用在 64 位明文上。最后一步是这个替代操作的逆操作。倒数第二步是交换左 32 位和右 32 位。剩下的 16 步在功能上完全相同，但使用了原始密钥的不同函数作为参数。DES 算法的设计允许使用同样的密钥来完成解密过程，这正是任何一个对称密钥算法必须满足的一个特性。在 DES 算法中，解密的步骤只是加密步骤的相反顺序而已。

这些中间步骤的操作如图 8-7(b)所示。每个步骤接受 2 个 32 位输入，并产生两个 32 位输出。左边的输出是右边输入的一份副本。右边的输出是左边输入与一个函数值逐位异或的结果，该函数的输入参数有两个，分别是右边的输入和当前步骤所用的密钥 K_i 。所有的复杂性都体现在这个函数中。

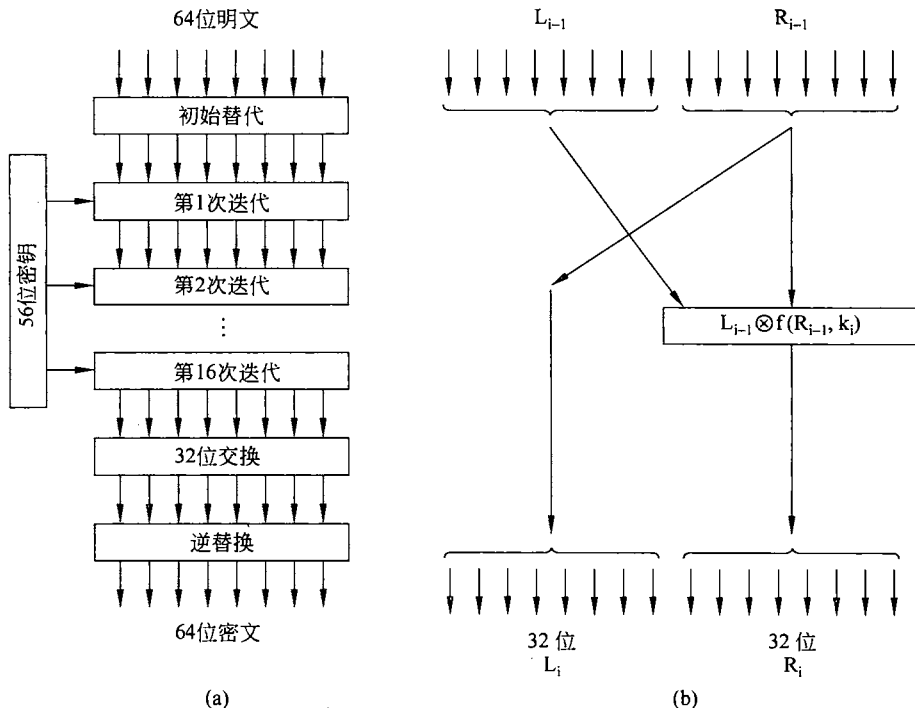


图 8-7 数据加密标准
(a) 一般结构; (b) 一次迭代的细节
 $\otimes$ 表示“异或”操作

此函数包含 4 个顺序执行的步骤。第一步，根据一个固定的替代和复制规则，将 32

位 R_{i-1} 扩展成一个 48 位的数字 E 。第二步, E 和 K_i 被异或在一起。然后, 异或的结果被分成 8 个 6 位组, 每个 6 位组被输入到一个不同的 S 盒中。对于每个 S 盒, 共有 64 种可能的输入, 每种输入被映射到一个 4 位输出上。最后, 将这 8×4 位通过一个 P 盒。

这 16 次迭代中的每一次迭代使用了不同的密钥。在算法开始之前, 先在 56 位的密钥上执行一个 56 位替代操作。在每一次迭代之前, 密钥被分成两个 28 位, 每个 28 位向左循环移位, 移动的位数取决于当前的迭代次数。移位以后再执行另一个 56 位替代即可导出 K_i 。在每一轮上, 从这 56 位中提取出不同的 48 位子集并对它进行排列。

有时候可以用一项特殊的技术来加强 DES 的强度, 这项技术称为白化 (whitening)。这项技术的做法是, 在将每一个明文数据块送给 DES 之前, 先用一个随机 64 位密钥对它执行异或操作, 然后在传输之前对 DES 输出的结果密文用第二个 64 位密钥执行异或操作。去掉白化也非常容易, 只要运行逆向操作即可 (前提条件是接收方也拥有这两个白化密钥)。由于这项技术有效地增加了密钥的长度, 所以, 它使得用穷举法来搜索密钥空间需要消耗更多的时间。请注意, 每个明文数据块使用同样的白化密钥 (即这里总共只有一个白化密钥)。

从 DES 被发布的那一天起, 有关 DES 的争议就一直不断。DES 建立在一个由 IBM 公司开发并拥有专利的加密算法基础之上, 该算法称为 Lucifer, 它使用了 128 位密钥, 而 DES 采用了 56 位密钥。当美国联邦政府希望为非机密的用途建立一个加密算法标准时, 它“邀请”IBM 公司与 NSA (美国政府中专门破解密码的机构)“讨论”有关的事项。NSA 雇佣了一支世界上最强大的数学家和密码学家队伍。NSA 是如此之神秘, 以至于在工业界流传着这样一个笑话:

问: NSA 代表什么?

答: 代表“不存在的机构”(No Such Agency)。

实际上, NSA 代表了国家安全局 (National Security Agency)。

经过讨论之后, IBM 公司将密钥从 128 位缩短到 56 位, 并且将 DES 的设计过程加以保密。许多人怀疑缩短密钥长度的原因是为了保证只有 NSA 能够破解 DES, 而经费预算较少的其他组织却无法破解 DES。这种不公开设计过程的做法使人们猜测 DES 算法中隐藏了一个后门, NSA 利用这个后门可以很容易地破解 DES。当 NSA 的一个雇员非常谨慎地通知 IEEE 取消一个计划好的密码学会议时, 更加重了人们的这种担心。但 NSA 否认这一切。

1977 年, 斯坦福大学的两位密码学研究人员 Diffie 和 Helman (1977) 设计了一台能破解 DES 的机器, 估计它的造价需要两千万美元。给定一小段明文和匹配的密文, 这台机器利用穷举法搜索整个密钥空间 (共有 2^{56} 个密钥), 可以在 1 天内找到密钥。现在, 这样一台机器的造价已经大大低于 100 万美元 (Kumar 等, 2006)。

三重 DES

1979 年初, IBM 公司意识到 DES 的密钥长度太短, 于是设计了一种方法, 利用三重加密来有效地增加密钥长度 (Tuchman, 1979)。所选择的方法如图 8-8 所示, 该方法后来已被集成到国际标准 8732 中。这种方法使用两个密钥, 共包含了 3 个步骤。第 1 步按照常规的方式用密钥 K_1 执行 DES 加密; 在第 2 步中, DES 以解密方式运行, 并使用 K_2 作为密

钥。最后，再次用 K_1 执行 DES 加密。

这种设计立即引发了两个问题。第一，为什么只使用两个密钥，而不是三个？第二，为什么使用 **EDE**（加密-解密-加密）模式，而不是 **EEE**（加密-加密-加密）呢？使用两个密钥的理由是，即使最为多疑的密码学家也认为，就目前而言，对于常规的商业应用来说 112 位已经足够了（对于密码学家来说，多疑往往被认为是一种个性，而不是缺陷）。将密钥长度增加至 168 位只会带来管理和传输另一个密钥的不必要开销，同时又没有任何实际的益处。

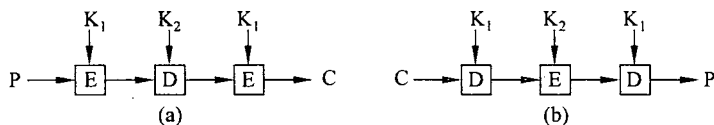


图 8-8

(a) 用 DES 进行三重加密；(b) 解密

采用“加密，解密，再加密”的理由是为了与现有的单密钥 DES 系统保持后向兼容性。DES 的加密和解密函数都是 64 位整数集之间的映射关系。从密码学的角度来看，这两个映射是同等强度的。然而，采用 EDE 而不是 EEE 之后，使用三重 DES 的计算机就可以与使用单密钥 DES 的计算机进行通话，做法很简单，只要设置 $K_1=K_2$ 即可。这种特性使得三重 DES 可以被逐步分阶段地应用到实际系统中，不过，学院派的密码学家对此并不感兴趣，但对于 IBM 公司和它的客户来说却非常重要。

8.2.2 AES——高级加密标准

随着 DES 逐渐走到生命的尽头，甚至三重 DES 也难以避免这样的厄运，美国国家标准及技术委员会（NIST, National Institute of Standards and Technology）决定美国政府需要一个新的密码标准作为非机密用途，NIST 是美国商务部的代理机构，承担着为美国联邦政府核定标准的重任。NIST 对所有关于 DES 的争议知道得非常清楚，它同时也知道，如果它直接宣布一个新的标准，则任何一个具有丰富密码学知识的人很自然会设想 NSA 已经在标准算法中内置了一个后门，因而 NSA 可以读取一切用该算法来加密的信息。在这样的情况下，可能没有人会使用此标准，它最终极有可能悄悄死去。

所以，NIST 采取了一种与政府官僚主义截然不同的做法：它发起了一场密码学比赛（竞赛）。1997 年 1 月，世界各地的研究人员接到邀请，希望为一个新的标准提交方案，这个新标准称为高级加密标准（AES, Advanced Encryption Standard）。比赛规则如下：

- (1) 算法必须是一个对称的块密码。
- (2) 所有的设计必须公开。
- (3) 必须支持 128、192、256 位密钥长度。
- (4) 软件实现和硬件实现必须都是可能的。
- (5) 算法必须是公有的，或者毫无歧视地授权给大众使用。

NIST 共收到了 15 个提案，于是它组织公开的会议展示这些提案，并积极鼓励参加会议的人寻找这些提案中的漏洞。1998 年 8 月，NIST 根据这些算法的安全性、效率、简单

性、灵活性和内存需求（对于嵌入式系统来说是非常重要的），选出 5 个算法进入最后的决赛。接下来又召开了更多的学术讨论会，同时也发生了更多的激烈论战。

2000 年 10 月，NIST 宣布 Joan Daemen 和 Vincent Rijmen 提出的 Rijndael 胜出。名称 Rijndael 是根据两位作者的姓氏（Rijmem+Daemen）合成的，其发音为 Rhine-doll（差不多是这样）。在 2001 年 11 月，Rijndael 成为 AES 美国政府标准，发布为 FIPS 197（联邦信息处理标准）。由于整个竞争过程绝对的开放，Rijndael 的技术特性以及获胜的小组是由两位年轻的比利时密码学家组成的（看上去不太可能为了迎合 NSA 而在算法内设置个后门），所以，Rijndael 已经变成世界上占主导地位的密码学密码。AES 的加密和解密现在是一些微处理器指令集的一部分（比如，Intel 公司）。

Rijndael 支持密钥长度和块的长度可以从 128 位一直到 256 位（以 32 位为间隔逐步向上递增）。密钥长度和块长度可以独立选择。然而，AES 规定块长度必须是 128 位，密钥长度必须是 128、192、256 位。尽管如此，很怀疑是否有人会使用 192 位密钥，所以，事实上，AES 提供两个变种：(1) 数据块为 128 位，密钥为 128 位；(2) 数据块为 128 位，密钥为 256 位。

在算法的处理中，我们将仅讨论 128/128 的情形，因为它有可能会变成商业规范。在 128 位的密钥空间中有 $2^{128} \approx 3 \times 10^{38}$ 个密钥。即使 NSA 想办法建造一台内含 10 亿个并行处理器的机器，并且每个处理器每微微秒就可以评估一个密钥，这样的机器也需要 10^{10} 年才能搜索完整个密钥空间。到那时候太阳的能量已经消耗殆尽了，因此人们只好点着蜡烛阅读解密结果了。

Rijndael

从数学的角度来看，Rijndael 是以伽罗瓦（Galois）域理论为基础的，Galois 域理论赋予 Rijndael 一些可以证明的安全特性。然而，我们也可以用 C 代码的形式来了解 Rijndael 算法，而不涉及数学理论。

如同 DES 一样，Rijndael 也使用置换和替代操作，并且它也使用了多轮策略。具体的轮数取决于密钥的长度和块的长度，对于 128 位密钥和 128 位块长度的情形，轮数为 10；对于更长的密钥或者更长的数据块，轮数可以增加到 14。然而，与 DES 不同的是，所有的操作都牵涉到整个字节，从而允许用硬件和软件高效地实现这些操作。Rijndael 的代码结构如图 8-9 所示。这个代码只是用作说明目的。安全代码的良好实现将遵从其他的实践规则，例如在使用后完全清楚敏感记忆。请参考（Ferguson 等，2010）中的例子。

函数 `rijndael` 有 3 个参数。它们分别是：`plaintext`（明文），一个包含了输入数据的 16 字节数组；`ciphertext`（密文），也是一个 16 字节的数组，它包含的是加密之后返回的输出结果；`key`（密钥），是 16 个字节长的密钥。在计算过程中，数据的当前状态被保存在一个字节数组 `state` 中，该数组的长度为 `NROWS×NCOLS`。对于 128 位数据块，该数组为 4×4 字节。利用这 16 字节可以存储整个 128 位数据块。

`state`（状态）数组首先被初始化为明文数据，然后在计算过程中每一步都要被修改。在有些步骤中，要执行字节对字节的置换；在其他的步骤中，这些字节被执行数组内的替代操作（即在数组内部重新排列顺序）。Rijndael 还用到了其他的变换。在算法的最后，`state` 中的内容被作为密文返回。


```

#define LENGTH 16          /* # bytes in data block or key */
#define NROWS4            /* number of rows in state */
#define NCOLS4            /* number of columns in state */
#define ROUNDS 10         /* number of iterations */
typedef unsigned char byte; /* unsigned 8-bit integer */
rijndael(byte plaintext[LENGTH], byte ciphertext[LENGTH], byte key[LENGTH])
{
    int r;                                /* loop index */
    byte state[NROWS][NCOLS];             /* current state */
    struct {byte k[NROWS][NCOLS];} rk[ROUNDS + 1]; /* round keys */
    expand_key(key, rk);                   /* construct the round keys */
    copy_plaintext_to_state(state, plaintext); /* init current state */
    xor_roundkey_into_state(state, rk[0]); /* XOR key into state */
    for (r = 1; r <= ROUNDS; r++) {
        substitute(state);                 /* apply S-box to each byte */
        rotate_rows(state);               /* rotate row i by i bytes */
        if (r < ROUNDS) mix_columns(state); /* mix function */
        xor_roundkey_into_state(state, rk[r]); /* XOR key into state */
    }
    copy_state_to_ciphertext(ciphertext, state); /* return result */
}

```

图 8-9 C 代码的 Rijndael 结构

在代码的开始处，首先将密钥扩展到 11 个与 state 同样长度的数组中。它们被保存在 rk 中，这里的 rk 是一个结构数组，其中每个结构元素包含一个状态数组。在这 11 个数组中，有一个被用在计算过程的开始处，其他 10 个被用在 10 轮计算中，每轮一个数组。从加密密钥（即原始密钥）导出每一轮密钥的过程非常复杂，因此我们这里不讨论这个过程。可以简单地这样来描述，轮密钥是通过反复地对密钥中不同的位组进行循环移位和异或操作而生成的。关于这些细节，请参考（Daemen 和 Rijmen, 2002）。

接下来的一步是将明文复制到 state 数组中，从而在后续的轮循环中得以处理。复制过程按照列的顺序进行，即前 4 个字节复制到第 0 列中，接下来 4 个字节复制到第 1 列中，以此类推。列和行都从 0 开始编号，但是轮数从 1 开始编号。图 8-10 显示了 12 个 4×4 大小的数组的初始化过程。

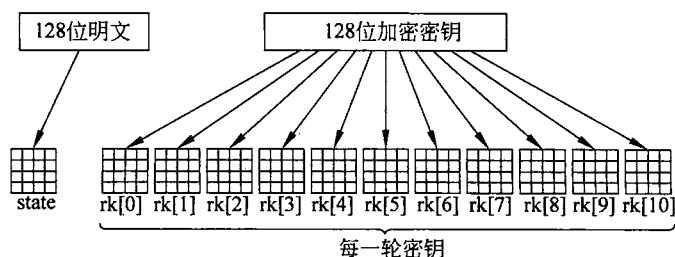


图 8-10 创建状态和 rk 数组

在主要计算过程开始之前还有一步：rk[0]被逐个字节地异或（XOR）到 state 中。换句话说，在 state 数组的 16 个字节中，每个字节都被它与 rk[0]中对应的字节异或之后的结果所取代。

现在该进入主循环了。该循环共执行 10 次迭代，每轮一次，并且在每一次迭代中都要变换 state。每一轮中的内容由 4 个步骤产生。第 1 步在 state 上执行一个逐字节的置换。按照顺序，每个字节被当作一个索引引用到一个 S 盒中，然后以该 S 盒中对应项的内容来替换该字节的值。这一步是一个直接的单字母表置换密码。与 DES 不同，DES 有多个 S 盒，但 Rijndael 只有一个 S 盒。

在第 2 步中，4 行中的每一行向左循环移动。第 0 行移动 0 字节（即不改变）；第 1 行左移 1 个字节；第 2 行左移 2 个字节；第 3 行左移 3 个字节。这一步起到的作用是将当前的数据内容扩散到整个块中，其效果类似于图 8-6 中的替代操作。

第 3 步独立地混合每一列，列与列之间并不相干。这个混合操作是通过矩阵乘法来完成的，在矩阵乘法中，新的列是老的列与一个常量矩阵的乘积，这里的乘法是有限 Galois 域上的乘法，即 $GF(2^8)$ 。虽然这听起来好像很复杂，但实际上存在一个算法可以用两个查表操作和三个 XOR 操作来计算新列的每个元素（Daemen 和 Rijndael, 2002，附录 E）。

最后，第 4 步将这一轮的密钥异或（XOR）到 state 数组中以便下一轮使用。

由于每一步都是可逆的，所以解密过程也很简单，只要反过来运行这个算法的每一步即可。然而，通过使用技巧，也可以用该加密算法来完成解密，只是用到的数据表不相同。

该算法的设计不仅具有极高的安全性，同时也达到极快的速度。在 2 GHz 的机器上，一个良好的软件实现应该可以达到 700 Mbps 的加密速率，这个速度足以实时地加密 100 部 MPEG-2 视频。当然，用硬件实现更快。

8.2.3 密码模式

尽管 AES 算法（或者 DES，或者其他任何块密码算法）如此复杂，但归根到底，它们本质上只是一种单字母置换密码算法，只不过使用了极大的字符（对于 AES 为 128 位字符，对于 DES 为 64 位字符）。任何时候当同样的明文块进入到算法前端时，同样的密文块就从后端输出。如果你用同样的 DES 密钥加密明文 abcdefgh 100 次，那么你就会得到同样的密文 100 次。入侵者可以充分发掘这种特性来协助攻破整个密码系统。

电码本模式

为了看清楚如何利用这种“单字母置换密码算法”的特性来局部破解密码算法，我们将使用（三重）DES 作为例子，因为描述 64 位数据块比 128 位数据块更容易，而且 AES 也有同样的问题。为了使用 DES 来加密一长段明文，最简单的做法是将它分割成连续的 8 字节（64 位）数据块，然后用同样的密钥逐个加密这些数据块。如果有必要的话，将最后一段明文填补至 64 位。这项技术称为电码本（ECB, Electronic Code Book）模式，它类似于老式代码簿。在老式的代码簿中，每个明文单词的旁边列出了它的密文（通常是 5 个十进制数字）。

如图 8-11 所示，我们从一个计算机文件开始，该文件列出了某一家公司已确定的奖励给雇员的年终奖金。文件由连续的 32 字节长的记录构成，每个雇员一条记录，正如图中所显示的，其格式为：16 字节的名字、8 字节的职位和 8 字节的奖金。现在利用（三重）DES 将图中显示的 16 个 8 字节块（从 0 到 15 编号）进行加密。

姓名	职位	奖金
A d a m s , L e s l i e	C l e r k	\$ 100,000
B l a c k , R o b i n	B o s s	\$ 500,000
C o l l i n s , K i m	M a n a g e r	\$ 100,000
D a v i s , B o b b i e	J a n i t o r	\$ 5

字节 ← 16 8 8

图 8-11 一个文件的明文被加密成 16 个 DES 块

Leslie 刚刚与老板有过冲突，因此他不期望会得到很多奖金。相反，Kim 则深得老板的喜爱，而且每个人都知道这一点。在这个文件被加密之后和它被送给银行之前，Leslie 有机会访问该文件。请问，在仅仅拿到加密文件的情况下，Leslie 有可能扭转这种不公平的局面吗？

这完全没有问题。Leslie 所需要做的事情是，将第 12 个密文块（这是 Kim 的奖金）做一份副本，然后用它替换掉第 4 个密文块（这是 Leslie 的奖金）。虽然 Leslie 并不知道第 12 个密文块到底包含了什么内容，但是他可以期望今年有一个愉快的圣诞节（复制第 8 个密文块也是可以的，但是极有可能被检测出来；另外，Leslie 并不是一个很贪婪的人）。

密码块链模式

为了对抗这种类型的攻击，所有的块密码算法可以按各种不同的方式链接起来。那样，Leslie 所做的密文块替换方法将导致在解密之后，从被替换的块开始的明文变成垃圾数据。一种链接方法是密码块链（cipher block chaining）。在这种方法中，如图 8-12 所示，每个明文块在被加密之前，先与上一个密文块执行一次异或操作。因此，同样的明文块不再被映射到同样的密文块上，加密操作也不再是一个大的单字母置换密码了。第一个块与一个随机选取的初始矢量（IV, Initialization vector）执行异或操作，该初始向量（以明文方式）随着密文一起被传输。

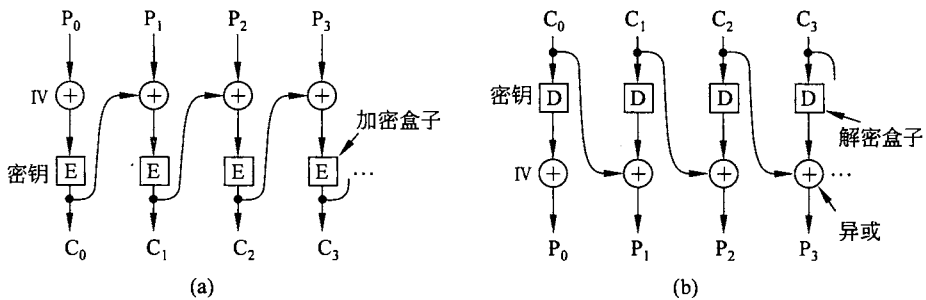


图 8-12 密码链接块

(a) 加密; (b) 解密

通过查看图 8-12 中的例子，我们可以看出密码块链模式的工作原理。首先我们计算 $C_0 = E(P_0 \text{ XOR } IV)$ ，然后计算 $C_1 = E(P_1 \text{ XOR } C_0)$ ，以此类推。解密过程使用 XOR 逆转整个过程，有 $P_0 = IV \text{ XOR } D(C_0)$ ，以此类推。请注意，第 i 块的加密操作是从 0 到 $i-1$ 块所有明文的一个函数，所以同样的明文根据它所出现的位置的不同将生成不同的密文。像 Leslie 所做的变换将导致从 Leslie 的奖金域开始的两个块变得没有任何意义。对于精明的安全检查官来说，这种特点可能正好暗示了应该从哪里开始进行深入的调查。

密码块链模式还有一个好处，即同样的明文块并不导致同样的密文块，这使得密码分析更加困难。实际上，这也正是它被广泛采用的原因。

密码反馈模式

然而，密码块链模式也有一个缺点，即只有当整个 64 位数据块到达以后才可以开始解密。对于逐字节的加密应用，可以使用如图 8-13 所示的密码反馈模式 (cipher feedback mode)，它使用了 (三重) DES。对于 AES，基本思想完全相同，只要使用 128 位移位寄存器。在这个图中，加密机正好处于“字节 0 至 9 已经被加密并发送出去”之后的状态。当明文字节 10 到来时，如图 8-13(a)所示，DES 算法被作用在 64 位移位寄存器上，并产生一个 64 位密文。该密文最左边的字节被提取出来，并且与 P_{10} 进行异或。然后该字节被传送到输出线路上。并且，移位寄存器左移 8 位，使得 C_2 被从最左边移走，而 C_{10} 被插入到 C_9 右边空出来的位置中。

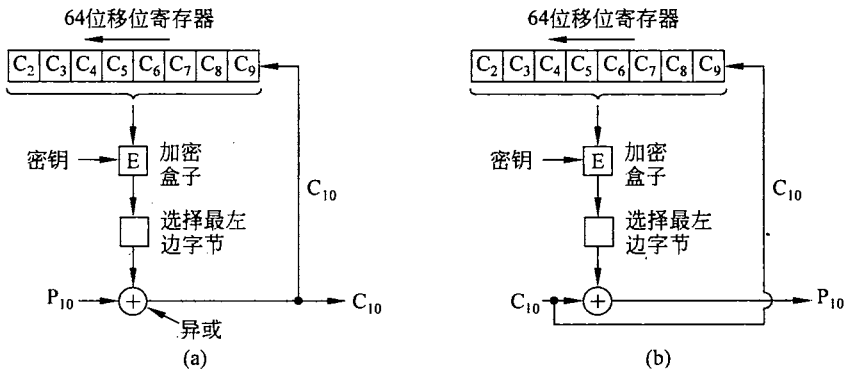


图 8-13 密码反馈模式
(a) 加密; (b) 解密

请注意，移位寄存器的内容依赖于明文的全部以前的历史，所以，若一个模式在明文中重复多次，则经过加密以后，在密文中每次出现都不相同。如同密码块链模式一样，密码反馈模式也需要一个初始向量来启动整个加密过程。

密码反馈模式的解密过程如同加密过程一样。尤其是，移位寄存器的内容是被加密，而不是被解密，所以，为了生成 P_{10} 而被选出来与 C_{10} 做异或的那个字节，与在加密过程中为了生成 C_{10} 而与 P_{10} 做异或的字节是同一个字节。只要两个移位寄存器保持不变，解密过程就不会错。图 8-13(b)显示了这一过程。

密码反馈模式的一个问题是，如果密文在传输过程中只有一位意外发生翻转，则在解密过程中，当坏字节位于移位寄存器中所波及的 8 个字节都将遭到破坏。一旦坏字节移出了移位寄存器以后，后面产生的明文仍然是正确的。因此，单位翻转错误只影响相对局部的区域，而不会破坏消息中其余剩下的部分，但是其影响的位数与移位寄存器的宽度相等。

流密码模式

然而对于一些应用来说，1 位传输错误会捣乱 64 位明文，这种影响还是太大了。对于这些应用来说，它们可以选择第四种方案，即流密码模式 (stream cipher mode)。它的工作

方式是用一个密钥来加密一个初始向量以便生成一个输出块；然后用同样的密钥对这个输出块进行加密得到第二个输出块；再对这一块进行加密以生成第三块，以此类推而进行加密。输出块的序列（可以任意长）称为密钥流（keystream），它就像一次性密钥那样与明文做异或操作来生成密文，如图 8-14(a)所示。请注意，初始矢量 IV 仅仅被用在第一步中；在此以后，输出数据块被加密。同时也要注意，密钥流与数据是独立的，所以，如果有必要的话，可以将密钥流提前计算出来，而且，它对于传输错误完全不敏感。解密过程如图 8-14(b)所示。

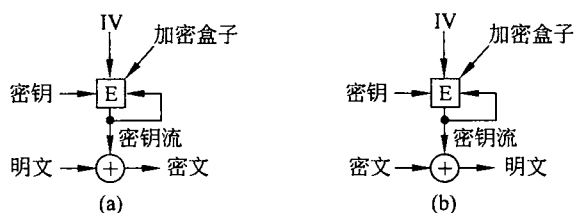


图 8-14 流密码模式
(a) 加密；(b) 解密

解密过程发生在接收端生成相同的密钥流。由于密钥流仅仅依赖于初始矢量 IV 和密钥，所以，它不会受到密文中传输错误的影响。因此，密文中的 1 位传输错误仅仅导致被解密出来的明文中产生同样的 1 位错误。

很关键的一点是，对于一个流密码，永远不要两次使用同样的（密钥，IV）对，因为一旦这样做以后，则每次都会生成同样的密钥流。两次使用同样的密钥流将导致密文受到密钥流重用攻击（keystream reuse attack）。假定明文块 P_0 经过一个密钥流加密之后得到 $P_0 \text{ XOR } K_0$ 。后来，第二个明文块 Q_0 也用同样的密钥流加密之后得到 $Q_0 \text{ XOR } K_0$ 。一个入侵者捕捉到这两个密文块以后，他只需将它们异或一下就可以得到 $P_0 \text{ XOR } Q_0$ ，从而消掉了密钥。现在入侵者有了两个明文块的 XOR 结果。如果其中一个明文被入侵者知道或者猜中的话，另一个明文自然也暴露无遗。无论如何，利用消息的统计特性，对两个明文流的 XOR 结果实施破解的可能性更大。例如，对于英语文本，在流中最常见的特征可能是两个空格的 XOR，其次是空格和字母“e”的 XOR，等等。简而言之，有了两个明文的 XOR 信息以后，密码分析者就有更多的机会来推断出这两个明文。

计数器模式

除了电码本模式以外其他所有模式都存在这样一个问题：要想随机访问加密之后的数据是不可能的。例如，假设通过网络传输一个文件，然后以密文的形式将文件存储在磁盘上。如果接收端计算机是一台有可能被偷走的笔记本电脑，那么这可能是一种非常合理的工作方式。使所有关键的文件都以加密的形式来保存可以极大地降低潜在的危险性，因为即使计算机落入敌手之后秘密信息也不易泄漏出去。

然而，磁盘文件通常是以非顺序的方式来访问的，特别是数据库中的文件。如果用密码块链模式来加密一个文件，那么为了访问一个随机的块，要求解密在它之前的所有数据块，而这是一个非常昂贵的操作。出于这个原因，后来又发明了另一种模式，即计数器模式（counter mode），如图 8-15 所示。在这里，明文并不直接被加密，相反，被加密的是初

始矢量加上一个常量的值，结果得到的密文再与明文做异或。通过“为每个新的数据块使初始向量递增 1”这种办法，在文件中任何地方的块都可以直接解密，而无须解密所有在它之前的数据块。

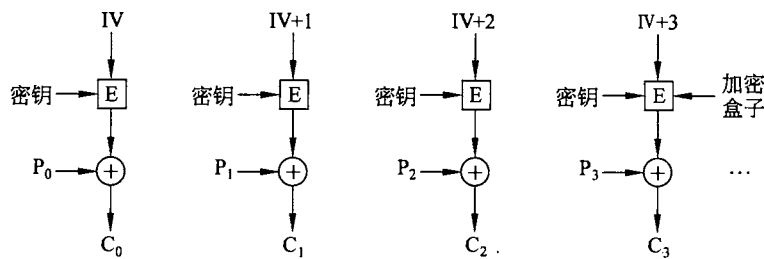


图 8-15 使用计数器模式的加密过程

尽管计数器模式非常有用，但它有一个弱点值得指出。假设同样的密钥 K 在将来又被使用了（明文不同，但 IV 相同），并且攻击者获得了这两次运行中所有的密文。这两种情形下的密钥流是相同的，从而使密码算法遭受密钥流重用攻击，我们在讨论流密码模式时曾经提到过这种类型的攻击。密码分析者只要将两个密文 XOR 在一起，就可以消除所有的密码保护，并获得明文的 XOR 结果。这个弱点并不意味着计数器模式的想法不好，而仅仅说明密钥和初始向量应该独立地随机选取。即使同样的密钥偶尔被用到了两次，如果这两次的 IV 不相同的话，则明文仍然是安全的。

8.2.4 其他密码模式

如果仅考虑可靠性，那么 AES (Rijndael) 和 DES 是众所周知的对称密钥密码学算法和业界选择的标准（如果你使用 AES 的产品而 AES 被破解，那么没有人会抱怨；但是如果你选用一个非标准密码而密码被破解，那么一定会有人怪你）。然而，值得说明的是已经有些其他对称密钥算法被实际使用。它们中的一些被嵌入到不同的产品。图 8-16 列出了少数几个比较常见的其他对称密钥。也可以使用组合密钥，例如，在使用 Twofish 加密后再用 AES 加密，必须破解两个密码才能恢复数据。

密码算法	作者	密钥长度	注释
DES	IBM	56位	太脆弱，现已不用
RC4	Ronald Rivest	1~2048位	小心：某些密钥很脆弱
RC5	Ronald Rivest	128~256位	好，但是专利
AES(Rijndael)	Daemen和Rijmen	128~256位	最好的选择
Serpent	Anderson、Biham、Knudsen	128~256位	很强大
Triple DES	IBM	168位	好，但年代久了
Twofish	Bruce Schneier	128~256位	很强大，被广泛使用

图 8-16 一些常见的对称密钥密码学算法

8.2.5 密码分析

在结束关于对称密码学主题的讨论之前，至少应该提及密码分析领域中的 4 个进展。

第一个进展是区分密码分析 (differential cryptanalysis) (Biham 和 Shamir, 1993)。这项技术可以被用来攻击任何一种块密码算法。它的工作原理是：首先用一对仅在少量的位上有差异的明文块来执行加密，然后仔细观察加密过程中内部每一次迭代所发生的情况。在许多情况下，有些位模式可能比其他的位模式更经常出现，这种现象可以导致概率攻击。

第二个值得提及的密码分析进展是线性密码分析 (linear cryptanalysis) (Matsui, 1994)。它只需用 2^{43} 个已知明文就可以破解 DES。它的工作原理是：将明文和密文中特定的位 XOR 在一起，然后检查结果。当重复这样的过程后，应该一半的位为 0，另一半的位为 1。然而，通常情况下密码算法会在某一个方向上引入偏差，不管这个偏差有多小，它总可以用来降低工作量。有关细节过程，请参考 Matsui 的论文。

第三个进展是通过对电子功率消耗的分析来发现秘密密钥。计算机通常使用 3 伏代表“1”位，用 0 伏来代表“0”位。因此，处理“1”比处理“0”需要更多的电能。如果一个密码算法由一个循环来完成的，并且这个循环按照顺序逐位地处理密钥中的位，那么，攻击者用一个慢的时钟（比如 100 Hz）来代替 n GHz 的主时钟，并且用弹簧夹夹住 CPU 的电源和地线就可以精确地监视每条机器指令所消耗的功率。利用这些数据来推断出密钥的做法竟然出奇地容易。这种密码分析方法很容易被挫败，只要用汇编语言小心地编写算法，以确保功率消耗与密钥独立，也与每一轮的密钥独立。

第四个进展是时间分析。密码算法往往充满了对轮密钥中的位进行测试的 if 语句。如果 then 和 else 部分的执行需要不同长度的时间，则只要减慢时钟，并且观察每一步所花的时间，就有可能推断出轮密钥。一旦获知了所有的轮密钥，原始的密钥就很容易被计算出来。还可以同时使用功率分析法和时间分析法，从而使破解工作更加容易。虽然功率分析和时间分析看起来好像是外来的（即不针对算法本身的特点），实际上它们的威力非常强大，足以破解任何一个未经专门设计来对抗这两种解密方法的密码算法。

8.3 公开密钥算法

在历史上，分发密钥往往是绝大多数密码系统中最薄弱的环节。不管一个密码系统有多强，如果入侵者能够偷取到密钥，则整个系统就变得毫无价值。密码学家总是认为加密密钥和解密密钥是一样的（或者很容易从一个推导出另一个）。但是，这个密钥必须要被分发给一个系统的所有用户。因此，看起来这里存在一个固有的问题：密钥必须被保护起来，以防被偷，但是它们又必须要被分发出去，所以，它们不可能仅仅被锁在银行的保险柜中。

1976 年，斯坦福大学的两位研究人员 Diffie 和 Hellman 提出了一种全新的密码系统，在这种密码系统中，加密密钥和解密密钥并不相同，而且不可能很轻易地从加密密钥推导出解密密钥。在他们的提案中，（受密钥控制的）加密算法 E 和解密算法 D 必须满足 3 个要求。这 3 个要求可以简述如下：

- (1) $D(E(P)) = P$ 。
- (2) 从 E 推断出 D 极其困难。
- (3) E 不可能被选择明文 (chosen plaintext) 攻击破解。

第一个需求意味着，如果我们将 D 作用在一条被加密的消息 $E(P)$ 上，则可以恢复原来

的明文消息 P 。如果没有这个特性,则合法的接收者就无法解密密文了。第二个需求不言而喻。第三个需求是必要的,因为正如我们马上将会看到的那样,入侵者有可能用此算法对他们的核心内容进行试验。具备了上述这些条件以后,加密密钥就有理由可以被公开了。

这种方法的工作过程如下所述。有一个人,比如说 Alice,希望接收秘密消息,她首先设计了两个满足以上要求的算法。然后,加密算法和 Alice 的密钥都被公开,因此该系统被命名为公开密钥密码系统 (public key cryptography)。例如, Alice 可以把公开密钥放在她的 Web 主页上。我们将使用标记 E_A 来表示以 Alice 的公开密钥作为参数的加密算法。类似地,以 Alice 的私有密钥作为参数的(秘密)解密算法被记为 D_A 。Bob 也做同样的事情,即公开 E_B ,但是 D_B 保密。

现在,我们来看是否能够在 Alice 和 Bob (以前两者从来没有联系过)之间建立一个安全的通道。假设 Alice 的加密密钥 E_A 和 Bob 的加密密钥 E_B 位于某些公开可读的文件中。现在, Alice 取出她的第一个消息 P ,并计算 $E_B(P)$,然后将它发送给 Bob。Bob 接收到消息以后,利用他的秘密密钥 D_B 对消息进行解密(即计算 $D_B(E_B(P)) = P$)。其他没有人能够读取已被加密的消息 $E_B(P)$,因为假设加密系统足够强,并且从公开已知的 E_B 推导出 D_B 是极其困难的。为了发送一个应答消息 R , Bob 传输 $E_A(R)$ 。现在 Alice 和 Bob 就可以安全地进行通信了。

这里提一下关于术语的说明。公开密钥密码系统要求每个用户拥有两把密钥:一个公开密钥(简称公钥)和一个私有密钥(简称私钥)。当其他人给某个用户发送加密消息时,他们使用该用户的公钥;当这个用户需要解密消息时,他(她)使用自己的私钥。我们在下文中将统一地把这两个密钥分别称为公钥和私钥,从而与传统的对称密钥密码系统中用到的秘密密钥区分开。

8.3.1 RSA

现在唯一的问题是,我们需要找到满足上述所有 3 个要求的算法。由于公开密钥密码系统的潜在优势,许多研究人员不遗余力地工作,陆续发表了一些满足要求的算法。一种比较好的方法是由 MIT 的一个小组发现的(Rivest 等, 1978),该方法的名字来源于 3 个发现者名字的首字母(Rivest、Shamir、Adleman): RSA。尽管 30 多年来,有大量的工作企图破解该方法,但它都抵抗住了,所以它被认为是一个非常强的公开密钥算法。实际上有大量的安全保障都建立在它的基础之上。正是这个原因, Rivest、Shamir 和 Adleman 共同被授予了 2002 年的 ACM 图灵奖。它的主要缺点是要想达到好的安全性,密钥至少需要 1024 位(相比之下,对称密钥算法只需 128 位),这也使得它的运算速度非常慢。

RSA 方法基于数论中的一些原理。我们现在简要地说明如何使用该方法,有关详细的信息请参考 Rivest 等人的论文。

- (1) 选择两个大的素数 p 和 q (典型情况下为 1024 位)。
- (2) 计算 $n=p \times q$ 和 $z=(p-1) \times (q-1)$ 。
- (3) 选择一个与 z 互素的数,将它称为 d 。
- (4) 找到 e , 使其满足 $e \times d = 1 \bmod z$ 。

提前计算出这些参数以后,我们就可以开始执行加密了。首先将明文(可以看作一个

位串)分成块,使得每个明文消息 P 落在间隔 $0 \leq P < n$ 中。为了做到这一点,只要将明文划分成 k 位的块即可,这里 k 是满足 $2^k < n$ 的最大整数。

为了加密消息 P ,只要计算 $C = P^e \pmod{n}$ 即可。为了解密 C ,只要计算 $P = C^d \pmod{n}$ 即可。可以证明,对于指定范围内的所有 P ,加密和解密函数互为反函数。为了执行加密,你需要 e 和 n ;为了执行解密,你需要 d 和 n 。因此,公钥是由 (e, n) 对组成的,而私钥是由 (d, n) 对组成的。

这种方法的安全性建立在分解大数的难度基础之上。如果密码分析者能够分解 n (公开已知的),那么他就可以找到 p 和 q ,从而得到 z 。有了 z 和 e 之后,只要使用欧几里得算法就能找到 d 。幸运的是,数学家们探索大数分解法至少有 300 多年了,几百年积累起来的经验表明这确实是一个极其困难的问题。

根据 Rivest 和同事们的研究工作,若使用穷举法,分解一个 500 位十进制数需要 10^{25} 年的时间。在两种情况下,他们假设使用了已知的最佳算法,并且计算机的指令时间为 1 微秒。使用 100 万个处理器并行计算,每个指令时间为 1 纳秒,仍然需要 10^{16} 年的时间。即使计算机的速度继续加快,以每 10 年一个数量级的速度增长,则仍然需要许多年才能使分解一个 500 位十进制数的方法变得实际可行,到那个时候,我们的后代们只要选择更大的 p 和 q 就可以了。

图 8-17 给出了一个简单的教学示范例子,它说明了 RSA 算法如何工作。在这个例子中,我们选择 $p=3$ 和 $q=11$,因此 $n=33$ 和 $z=20$ 。由于 7 和 20 没有公因子,所以 d 值取 $d=7$ 是合适的。有了这些选择以后,通过解方程 $7e \equiv 1 \pmod{20}$ 可以找到 e ,结果是 $e=3$ 。针对明文消息 P 的密文 C 可以通过 $C = P^3 \pmod{33}$ 求得。接收方在解密密文时,只要利用 $P = C^7 \pmod{33}$ 规则即可。图中的例子显示了明文“SUZANNE”的加密过程。

明文(P)		密文(C)			解密后	
符号	数值	P^3	$P^3 \pmod{33}$	C^7	$C^7 \pmod{33}$	符号
S	19	6859	28	13 492 928 512	19	S
U	21	9261	21	1 801 088 541	21	U
Z	26	17 576	20	1 280 000 000	26	Z
A	01	1	1	1	01	A
N	14	2744	5	78 125	14	N
N	14	2744	5	78 125	14	N
E	05	125	26	8 031 810 176	05	E

发送方计算

接收方计算

图 8-17 RSA 算法的一个例子

因为这个例子中选择的素数太小,以至于 P 必须小于 33,因此每个明文块只能包含一个字符。其结果是一个单字母置换,让人觉得该方法并无稀奇之处。然而,如果我们选择 p 和 $q \approx 2^{512}$,则我们有 $n \approx 2^{1024}$,每个块就可以达到 1024 位,或者 128 个 8 位字节,相比之下,DES 的数据块是 8 个字符,AES 的块是 16 个字节。

应该指出,像以上描述的 RSA 用法有点类似于按 ECB 模式使用对称密钥算法的做法——即,同样的输入块得到同样的输出块。因此,我们需要使用某种链接形式来加密数据。然而,实际上大多数基于 RSA 的系统主要利用公开密钥算法来分发一次性会话密钥,再将这些会话密钥用于某一个对称密钥算法,比如 AES 或者三重 DES。若真要用 RSA 来

加密大量的数据，则运算速度太慢，因此 RSA 被广泛用于密钥的分发。

8.3.2 其他公开密钥算法

虽然 RSA 已经得到广泛使用，但它并不是唯一的已知公开密钥算法。第一个公开密钥算法是背包算法（Merkle 和 Hellman, 1978）。背包算法的思想是：某个人拥有大量的物品，每个物品的重量各不相同。他秘密地选择其中一组物品并将这些物品放到一个背包中来编码一个消息。背包中物品的总重量被公开，所有可能的物品和它们对应的重量也被公开。但是，背包中物品的明细则是保密的。在特定的附加限制条件下，“根据给定的总重量找出可能的物品明细列表”这个问题被认为是一个计算上不可行的问题，从而构成了此公开密钥算法的基础。

算法的发明者 Ralph Merkle 非常确信这个算法不可能被攻破，所以他悬赏 100 美元奖金给破解算法的人。Adi Shamir（即 RSA 中的“S”）迅速地破解了算法，并领走了奖金。Merkle 并没有气馁，他又加强了算法，并悬赏 1000 美元奖金给破解新算法的人。Ronald Rivest（即 RSA 中的“R”）也迅速地破解了新算法，并领取了奖金。Merkle 不敢再为下一个版本悬赏 10 000 美元奖金了，所以“A”（Leonard Adleman）就无法那样幸运也有机会破解它。不管怎么样，背包算法已经不再被认为是安全的，实际上也没有被采用。

其他一些公开密钥方案建立在计算离散对数的难度基础之上。（El Gamal, 1985）和（Schnorr, 1991）发明了使用这种原理的公开密钥算法。

另外还存在一些方案，比如基于椭圆曲线的公开密钥算法（Menezes 和 Vanstone, 1993），但是最主要的两大类算法是：建立在“分解大数的难度”基础上的算法，以及建立在“以大素数为模来计算离散对数的难度”基础上的算法。这些问题被认为的确很难解决，因为数学家们已经为之钻研了许多年而未能有所突破。

8.4 数字签名

许多法律的、金融的和其他文档的真实性是依据是否具有某一个授权人物的手写签名。影印件是无效的。为了能够用计算机化的消息系统来代替纸和笔墨文档的物理传输系统，必须要找到一种方法对文档进行签名，并且文档的签名不可被伪造。

设计一个代替手写签名的方案非常困难。基本上，我们需要的是这样一个系统，其中一方向另一方发送的签名消息必须满足以下条件：

- （1）接收方可以验证发送方所声称的身份。
- （2）发送方以后不能否认该消息的内容。
- （3）接收方不可能自己编造这样的消息。

第一个要求是必需的，例如在金融系统中，当顾客的计算机向银行的计算机订购一吨黄金时，银行的计算机必须要确信发出订单的计算机真正属于付款账号的所属公司。换句话说，银行必须要认证顾客的身份（顾客也要认证银行的身份）。

第二个要求也是必需的，它可以保护银行不被欺骗。假设银行买进了一吨黄金，之后

黄金的价格立刻跌了下来。一个不诚实的顾客可以起诉银行，宣称自己从来没有发出过购买黄金的订单。当银行在法庭上出示这条消息的时候，顾客否认自己曾经发送过此消息。这种“契约的任何一方以后不能否认自己曾经签过名”的特性称为不可否认性(nonrepudiation)。我们现在将要学习的数字签名方案可以用来提供这样的特性。

第三个要求也是必需的，它可以保护顾客的利益不受损害，例如，当黄金的价格暴涨时，银行企图构造一个“顾客要求购买一条黄金（而不是一吨黄金）”的签名消息。在这种欺骗的情形下，银行就可以保留剩余的黄金。

8.4.1 对称密钥签名

数字签名的一种做法是设立一个人人都信任并且又熟知一切的中心权威机构，比如 Big Brother (简称 BB)。每个用户选择一个秘密密钥，并且亲手将它送到 BB 的办公室。因此，只有 Alice 和 BB 才知道 Alice 的秘密密钥 K_A ，如此等等。

当 Alice 想要给她的银行业务员 Bob 发送一条签名的明文消息 P 时，她生成 $K_A(B, R_A, t, P)$ ，这里 B 是 Bob 的标识， R_A 是 Alice 选择的一个随机数， t 是一个时间戳（可用来保证该消息是最新的）， $K_A(B, R_A, t, P)$ 表示用她的密钥 K_A 加密之后的消息。然后，她按照图 8-18 所示的方式将该消息发送出去。BB 看到该消息来自 Alice，于是解密该消息，并按图中所示给 Bob 发送一条消息。给 Bob 的消息包含了 Alice 的原始消息的明文 P 和一条经过签名的消息 $K_{BB}(A, t, P)$ 。Bob 现在执行 Alice 的请求。

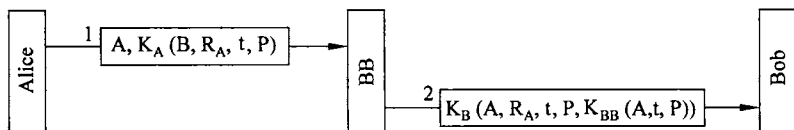


图 8-18 使用 BB 的数字签名

如果 Alice 后来拒绝承认自己曾经发送过该消息，那该怎么办呢？首先，人人都可以起诉别人（至少在美国是这样的）。最后，当案子到了法庭上，Alice 顽固地否认自己曾经给 Bob 发送过这条有争议的消息时，法官会问 Bob，他如何能保证这条有争议的消息确实来自 Alice 而不是 Trudy。Bob 首先指出，BB 在看到来自 Alice 的消息时，除非该消息是用 K_A 加密的，否则 BB 是不会接受该消息的，所以，Trudy 向 BB 发送一条谎称来自 Alice 的消息却没有被 BB 检测到，这是不可能的。

然后 Bob 在众目睽睽下出示了证据 A: $K_{BB}(A, t, P)$ 。Bob 说，这是一条由 BB 签名的消息，它可以证明 Alice 曾经发送 P 给 Bob。然后，法官请 BB（人人都信任 BB）解密证据 A。当 BB 证实了 Bob 说的是真话时，法官做出有利于 Bob 的判决。案子最终结束。

在图 8-18 的签名协议中，有个潜在的问题，那就是 Trudy 可能会重放其中某一条消息。为了使这个问题的危险性降低到最小，可以在每一个环节上用时间戳进行控制。而且，Bob 可以检查所有最近接收到的消息，查看这些消息是否使用过 R_A 。如果确有消息用到了 R_A ，则他可以将新收到的消息当作重放消息而丢弃。请注意，利用时间戳机制，Bob 将拒绝接收那些很老的消息。为了对付短时间内的重放攻击，Bob 只要检查每条入境消息中的 R_A ，以此判断在过去的一小时内是否曾经收到过来自 Alice 的这条消息。如果没有，则 Bob 就

可以安全地认为这是一个新的请求。

8.4.2 公开密钥签名

利用对称密钥密码技术来实现数字签名存在一个结构性问题：每个人都必须同意 BB 是可相信的。而且，BB 能够解读所有签名的消息。从逻辑来看，最有可能运行 BB 服务器的候选机构是政府、银行、会计事务所和律师事务所。不幸的是，老百姓对所有这些组织并不能寄予足够的信任。因此，若能在签名文档的时候不要求通过一个可信的权威机构就好了。

幸运的是，公开密钥密码学为这个领域做出了重要的贡献。我们假设公开密钥的加密算法和解密算法除了具有常规的 $D(E(P)) = P$ 属性以外，还具有 $E(D(P)) = P$ 属性（RSA 有这样的属性，所以这个假设并非不合理）。假设这种情况是成立的，那么 Alice 可以通过传输 $E_B(D_A(P))$ 向 Bob 发送一条签名的明文消息 P。请注意，Alice 知道她自己的（私有）密钥 D_A 以及 Bob 的公开密钥 E_B ，所以，Alice 能够这样做。

当 Bob 收到这条消息时，他像往常一样利用自己的私钥对消息做替代，从而得到 $D_A(P)$ ，如图 8-19 所示。他将这份信息放在一个安全的地方，然后通过使用 E_A 可得到原始的明文。

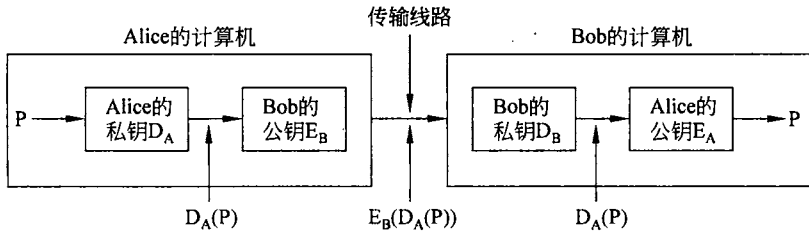


图 8-19 使用公开密钥密码数字签名

为了看清这种签名特性的工作原理，不妨假设 Alice 后来否认自己曾经给 Bob 发送过消息 P。当这个案子被提到法庭上时，Bob 可以同时出示 P 和 $D_A(P)$ 。法官很容易验证 Bob 是否真的拥有一条由 D_A 加密的消息，他只需简单地在消息上应用 E_A 即可。由于 Bob 并不知道 Alice 的私钥是什么，所以，Bob 能获得由 Alice 私钥加密的消息的唯一途径是 Alice 给他发送了这样的消息。当 Alice 因为作伪证和欺骗罪而被监禁起来时，她有足够的时间来设计新的有趣的公开密钥算法。

尽管用公开密钥密码技术实现数字签名是一种非常优雅的方案，但是，这里还存在一些涉及运行环境的问题，与基本算法无关。首先，只要 D_A 仍然是保密的，Bob 就可以证明一条消息确实是 Alice 发送的。如果 Alice 泄漏了她的密钥，那么这个论点就不再成立，因为任何人都可以发送这样的消息，包括 Bob 自己在内。

这时候就可能出现这样的问题，如果 Bob 是 Alice 的股票经纪人，假设 Alice 告诉 Bob 购买一支特定的股票或者国库券。很快地，股票价格急剧下降。Alice 为了否认她曾经给 Bob 发送过消息，她跑到警察局声称她家遭到抢劫，保存密钥的个人电脑被偷了。根据她所在州或者国家的法律，她可能需要，也可能不需要承担法律责任，尤其是如果她声称是在下班以后回到家里时才发现被抢了（即发生抢劫几小时以后才发现）。

与这种签名方案有关的另一个问题是，如果 Alice 决定要改变她的密钥，那该怎么办

呢？很显然，这样做是合法的，而且定期改变密钥可能是一种很好的做法。如果后来发生了法律纠纷，就像上面描述的那样，那么，法官将当前的 E_A 作用在 $D_A(P)$ 上，发现其结果并不等于 P 。这时候 Bob 就会很难堪。

原则上，任何一种公开密钥算法都可以用作数字签名。事实上的业界标准是 RSA 算法。许多安全产品都使用了 RSA 算法。然而，1991 年，NIST 建议使用 El Gamal 公开密钥算法的一个变种作为它们新的数字签名标准（DSS, Digital Signature Standard）。El Gamal 算法的安全性建立在计算离散对数的难度基础之上，而并非建立在分解大数的困难度基础上。

通常情况下，当政府试图推行一个密码学标准时，它总会招致各种非议。DSS 也受到了下述批评：

- (1) 太神秘（这是 NSA 设计的协议，它使用了 El Gamal 算法）。
- (2) 太慢（在检查签名时比 RSA 慢了 10~40 倍）。
- (3) 太新（El Gamal 算法还没有被完全分析透彻）。
- (4) 太不安全（固定的 512 位密钥）。

在后来的修订版本中，允许密钥长度增加到 1024 位，因此第（4）点已不成问题，然而前两点批评还是没有改变。

8.4.3 消息摘要

对签名方法的一个批评是，它们通常将两种不相同的功能耦合在一起：认证和保密。通常情况下，认证是必要的，但是保密性并不一定是必需的。而且，如果一个系统只提供认证而没有提供保密性的话，往往更加容易申请出口许可。下面介绍具备认证不要求加密整条消息的方案。

这个方案以单向散列函数思想为基础，这里的单向散列函数接受一个任意长度的明文作为输入，并且根据此明文计算出一个固定长度的位串。这个散列函数 MD 通常称为消息摘要（message digest），它有 4 个重要的特性：

- (1) 给定 P ，很容易计算 $MD(P)$ 。
- (2) 给定 $MD(P)$ ，要想有效地找到 P 是不可能的。
- (3) 在给定 P 的情况下，没有人能够找到满足 $MD(P') = MD(P)$ 的 P' 。
- (4) 在输入明文中即使只有 1 位的变化，也会导致完全不同的输出。

为了满足第 3 条，散列结果应该至少 128 位，最好还能更长一些。为了满足第 4 条，散列结果必须彻底弄乱明文中的位，和我们在对称密钥加密算法中看到的没有什么不同。

从一段明文计算出一个消息摘要必须比用公开密钥算法来加密这段明文要快得多，所以消息摘要可以被用来加速数字签名算法。为了看清楚这个工作过程，请再次考虑图 8-18 的签名协议。BB 现在不再利用 $K_{BB}(A, t, P)$ 作为 P 的签名，而是计算消息摘要，他将 MD 作用在 P 上，得到 $MD(P)$ 。然后 BB 用 $K_{BB}(A, t, MD(P))$ 代替原来的 $K_{BB}(A, t, P)$ ，作为他发送给 Bob 的消息（被 K_B 加密）中的第 5 项。

如果发生纠纷，Bob 可以出示 P 和 $K_{BB}(A, t, MD(P))$ 。当 BB 根据法官的要求将后者解密出来后，Bob 有了 $MD(P)$ ，以及他所宣称的 P 。这里 $MD(P)$ 保证是真实的。然而，由于 Bob 不可能有效地找到另一条具有相同散列结果的消息，所以法官很容易就可以确信 Bob

讲的是事实。按照这种方式来使用消息摘要可以节省加密时间和消息传输的开销。

消息摘要也可以用在公开密钥密码系统中，如图 8-20 所示。在这里，Alice 首先计算明文的摘要，然后她针对摘要进行签名，并且将签名之后的摘要与明文本身一起发送给 Bob。如果 Trudy 中途偷换掉 P，那么当 Bob 计算 MD(P)时就可以发现这一点。

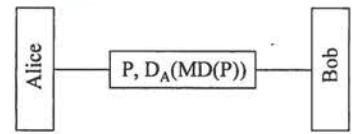


图 8-20 使用消息摘要的数字签名

SHA-1 和 SHA-2

各种消息摘要函数被提了出来。一个被广泛应用的消息摘要函数是安全散列算法 1 (SHA-1, Secure Hash Algorithm 1) (NIST, 1993)。像所有的消息摘要一样，它使用非常复杂的方法来弄乱位，以至于它的每个输出位都受到输入位的影响。SHA-1 由 NSA 开发并得到 NIST 的赏识，并被标准化在 FIPS 180-1 中。它按照 512 位的块大小来处理输入数据，并且生成一个 160 位的消息摘要。图 8-21 所示为一种典型的使用方式，Alice 用来发给 Bob 非保密性、但有她签名的消息。在此，她的明文消息被送入到 SHA-1 算法中，以便得到一个 160 位的 SHA-1 散列值。然后，她用自己的 RSA 私钥对散列值进行签名，并且将明文消息和签过名的散列值发送给 Bob。

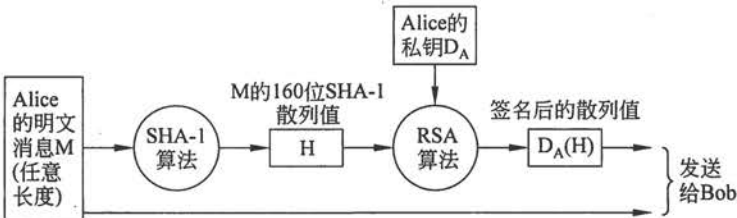


图 8-21 非保密性消息使用 SHA-1 和 RSA 签名

Bob 在接收到消息以后，他自己计算 SHA-1 散列值，并且利用 Alice 的公钥来解密签过名的散列值，从而得到原始的散列值 H。如果这两者一致的话，则认为该消息是有效的。因为对于 Trudy 来说，她无法在传输过程中既要修改（明文）消息，也要使修改之后的消息散列到同一个 H 上，所以，Bob 可以很容易地检测到 Trudy 对于消息 P 所做的任何改变。对于那些完整性很重要但内容并不需要保密的消息，广泛使用了图 8-21 所示的方案。那些计算量相对较小的计算，采用这种方案保证了在传输过程中对明文消息所做的任何修改都将（以极高的概率）被检测到。

现在我们大致看一下 SHA-1 的工作原理。首先它在消息的末尾填补一个 1 位，然后跟上一定数量的 0 位，0 位的个数必须保证填补之后的消息长度为 512 位的倍数。然后构造一个包含了填补之前消息长度的 64 位整数，并将它与消息的低 64 位进行或（OR）操作。在图 8-22 中，填补的位被显示在消息的右侧，因为英语文本和图片按照从左至右的顺序来书写或绘制（即右下角往往被认为是图片的尾部）。对于计算机来说，这种方向特性对应于 big-endian 字节序的机器，比如 SPARC，但是，SHA-1 总是填补在消息的末尾，而不管当前机器使用了哪一种字节序。

在计算过程中，SHA-1 维护了 5 个 32 位变量，从 H₀ 至 H₄，最后散列的结果就放在这些变量中。图 8-22(b)显示了这些变量。它们的初始常量由标准定义。

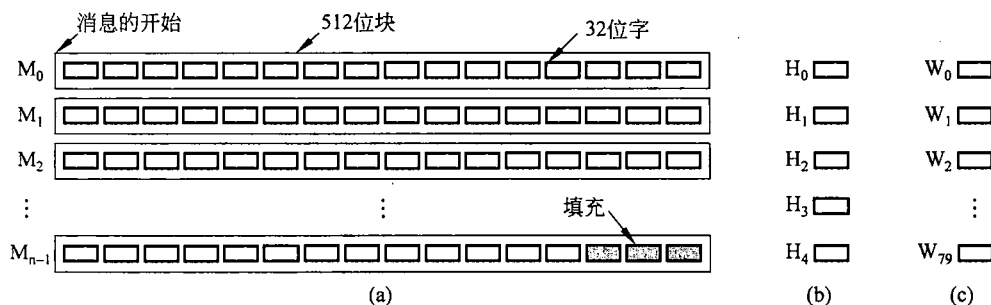


图 8-22

(a) 消息被填补成 512 位的倍数; (b) 输出变量; (c) 字数组

现在按照顺序逐个处理数据块 M_0 至 M_{n-1} 。对于当前的块, 首先将 16 个字复制到一个如图 8-22(c) 所示的 80 字的辅助数组 W 的前面部分。然后, 利用下面的公式来填充 W 中其他的 64 个字:

$$W_i = S^1(W_{i-3} \text{ XOR } W_{i-8} \text{ XOR } W_{i-14} \text{ XOR } W_{i-16}) \quad (16 \leq i \leq 79)$$

在这里 $S^b(W)$ 代表了 32 位字 W 循环左移 b 位。现在, 利用 H_0 至 H_4 分别初始化 5 个临时变量 (从 A 至 E)。

实际的计算过程可以用下面的伪 C 代码来表达:

```
for (i=0; i<80; i++){
    temp=S5(A)+fi(B,C,D)+E+Wi+Ki;
    E=D; D=C; C=S30(B); B=A; A=temp;
}
```

在此常量 K_i 由标准定义, 而混合函数 f_i 的定义如下:

$$f_i(B, C, D) = (B \text{ AND } C) \text{ OR } (\text{NOT } B \text{ AND } D) \quad (0 \leq i \leq 19)$$

$$f_i(B, C, D) = B \text{ XOR } C \text{ XOR } D \quad (20 \leq i \leq 39)$$

$$f_i(B, C, D) = (B \text{ AND } C) \text{ OR } (B \text{ AND } D) \text{ OR } (C \text{ AND } D) \quad (40 \leq i \leq 59)$$

$$f_i(B, C, D) = B \text{ XOR } C \text{ XOR } D \quad (60 \leq i \leq 79)$$

当循环中的 80 次迭代完成以后, A 至 E 被分别加到 H_0 至 H_4 上。

现在第一个 512 位的块处理完毕, 下一个块又接着开始处理。 W 数组又被按照新的块进行初始化, 但是, H 数组仍然不变。当这一块完成以后, 下一块又开始, 如此周而复始, 直到所有的 512 位消息块都被处理完毕。当最后一块被处理完以后, H 数组中的 5 个 32 位字即被输出作为 160 位的密码学散列值。RFC 3174 给出了 SHA-1 算法的完整 C 代码。

一些生成 256、384 和 512 位散列值的 SHA-1 新版本已经被开发出来。这些版本统称为 SHA-2。新版本不仅散列值比 SHA-1 散列值长, 而且摘要函数也得到修改, 弥补了一些 SHA-1 的潜在弱点。目前 SHA-2 还没有被广泛使用, 但是它未来的前景很被看好。

MD5

为了完整性, 我们介绍另一个很普及的消息摘要算法 MD5 (Rivest, 1992)。它由 Ronald Rivest 设计, 在他设计的一系列消息摘要算法中排在第五位。简单地说, 消息被填补到一个 448 位长 (modulo 512), 然后附加 64 位整数的原始消息长度, 使得总输入长度是 512

位的倍数。每一轮的计算都用一个正在运行的 128 位缓冲区，完全地混合一个 512 位的输入块。为何便于测量，混合时使用了一个用正弦函数构建的表。使用已知函数的目的是为了让人怀疑设计师留下一个巧妙但只有他们可以进入的后门。这个过程一直进行到所有的输入块都已经被用完。128 位缓冲区里的内容就构成了消息摘要。

经过了超过 10 年的实际使用和研究后，MD5 的弱点导致人们能够找到具有相同散列的冲突或不同的消息（Sotirov 等，2008）。这对于消息摘要来说是个丧钟，因为它意味着摘要不能安全地用来代表消息。因此，安全社团认为 MD5 将被破解，在可能的情况下应该换掉它，而且新的系统在设计时不应该再使用它。不过，你可能会看到一些已有系统仍然在使用 MD5。

8.4.4 生日攻击

在密码学领域，没有一事情会像表面上看到的那么简单。有人可能会认为，为了攻破一个 m 位的消息摘要，将需要 2^m 数量级的操作次数。事实上，（Yuval，1979）在他的论文“*How to Swindle Rabin*”中发表了一种生日攻击方法，用这种方法通常只需要 $2^{m/2}$ 量级的操作次数，现在他的这篇论文已经成为经典。

这种攻击的思想来源于数学教授们通常在他们的概率课堂上采用的一种技术。所提的问题是：一个班级中有多少个学生时，使得这个班级中两个学生具有相同生日的概率超过 $1/2$ ？大多数学生都认为答案将会超过 100。事实上，根据概率论，答案只是 23。如果不给出严密的分析，仅仅从直觉上，我们可以看到，在有 23 个学生的情况下，我们可以组成 $(23 \times 22)/2 = 253$ 对不同的组合，每一对组合有 $1/365$ 的命中概率。照这样看来，这个答案实际上就不那么令人惊奇了。

推广至更为一般的情形，如果在 n 个输入（人、消息等）和 k 个可能的输出（生日、消息摘要等）之间存在某一种从输入到输出的映射关系，那么就有 $n(n-1)/2$ 个输入对。若 $n(n-1)/2 > k$ ，则至少有一个匹配的机会是非常大的。因此，近似地，存在一个匹配的条件是 $n > \sqrt{k}$ 。这个结果意味着只要生成大约 2^{32} 条消息，然后在这些消息中寻找具有相同消息摘要的两条消息就有可能攻破一个 64 位的消息摘要。

现在来看一个实际的例子。州立大学的计算机科学系有一个终生教员的职位，并且有两个候选人：Tom 和 Dick。由于 Tom 比 Dick 早雇用两年，所以他是首先被考察的对象。如果他通过了，则 Dick 就没有机会。Tom 知道系行政主管 Marilyn 非常欣赏他的工作，所以他请 Marilyn 为他写一封推荐信给系主任，因为最终的决定权在系主任手中。一旦发出去之后，所有的信件就都是保密的。

Marilyn 告诉她的秘书 Ellen 给系主任写封信，并大略地描述了她在信中要表达的内容。当 Ellen 写完信后，Marilyn 将会检查一遍，并计算和签署 64 位摘要，然后发送给系主任。Ellen 可以稍后通过电子邮件发送这封信。

对于 Tom 来说很不幸的是，Ellen 正迷恋着 Dick，因而她想要陷害 Tom，于是，她写了下面这封具有 32 个用括号括起选项的信：

Dear Dean Smith,

This [letter | message] is to give my [honest | frank] opinion of Prof. Tom Wilson, who is [a candidate | up] for tenure [now | this year]. I have [known | worked with] Prof. Wilson for [about | almost] six years. He is an [outstanding | excellent] researcher of great [talent | ability] known [worldwide | internationally] for his [brilliant | creative] insights into [many | a wide variety of] [difficult | challenging] problems.

He is also a [highly | greatly] [respected | admired] [teacher | educator]. His students give his [classes | courses] [rave | spectacular] reviews. He is [our | the Department's] [most popular | bestloved] [teacher | instructor].

[In addition | Additionally] Prof. Wilson is a [gifted | effective] fund raiser. His [grants | contracts] have brought a [large | substantial] amount of money into [the | our] Department. [This money has | These funds have] [enabled | permitted] us to [pursue | carry out] many [special | important] programs, [such as | for example] your State 2000 program. Without these funds we would [be unable | not be able] to continue this program, which is so [important | essential] to both of us.

I strongly urge you to grant him tenure.

对于 Tom 来说, 不幸的是 Ellen 撰写并输入这封信后, 她还马上写了第二封信:

Dear Dean Smith,

This [letter | message] is to give my [honest | frank] opinion of Prof. Tom Wilson, who is [a candidate | up] for tenure [now | this year]. I have [known | worked with] Tom for [about | almost] six years. He is a [poor | weak] researcher not well known in his [field | area]. His research [hardly ever | rarely] shows [insight in | understanding of] the [key | major] problems of [the | our] day.

Furthermore, he is not a [respected | admired] [teacher | educator]. His students give his [classes | courses] [poor | bad] reviews. He is [our | the Department's] least popular [teacher | instructor], known [mostly | primarily] within [the | our] Department for his [tendency | propensity] to [ridicule | embarrass] students [foolish | imprudent] enough to ask questions in his classes.

[In addition | Additionally] Tom is a [poor | marginal] fund raiser. His [grants | contracts] have brought only a [meager | insignificant] amount of money into [the | our] Department. Unless new [money is | funds are] quickly located, we may have to cancel some essential programs, such as your State 2000 program. Unfortunately, under these [conditions | circumstances] I cannot in good [conscience | faith] recommend him to you for [tenure | a permanent position].

现在 Ellen 利用她的计算机彻夜地计算出每封信的 2^{32} 个消息摘要。第一封信的消息摘要有可能跟第二封信的消息摘要匹配。如果没有的话, 她可以增加一些选项然后今晚再尝

试一次。现在假设她找到了一个匹配。我们将“好的”信称为 A，“坏的”信称为 B。

Ellen 现在以电子邮件的方式将信 A 发送给 Marilyn 审核。同时将信 B 完全保密，不让任何人看到。Marilyn 当然批准了它，计算她的 64 位消息摘要，并且签了摘要，然后将签过名的摘要发送给系主任 Smith。而 Ellen 则单独地将信 B 以电子邮件方式寄送给系主任(请注意，她寄送的并不是 Marilyn 所想象的信 A)。

系主任在得到了信和签过名的消息摘要后，他针对信 B 运行消息摘要算法，发现与 Marilyn 送给他的摘要一致，于是解雇了 Tom。系主任并没有意识到 Ellen 生成了两封具有相同消息摘要的信，并且发送给他的信与 Marilyn 看到并批准的信不是同一封（一个选择性的结尾是：Ellen 将自己所做的一切全部告诉了 Dick。Dick 非常震惊，并与她断绝了关系。Ellen 也非常气愤，向 Marilyn 坦白了一切。Marilyn 打电话告诉了系主任，最终 Tom 获得了职位）。使用 SHA-1，生日攻击很难实施，因为即使每秒钟产生 10 亿个摘要，那也需要花 32 000 年来计算两封信的所有 2^{80} 个摘要（每封信有 80 种变化形式），而且即使这样还不能保证总会存在一个匹配。当然，如果使用 1 000 000 个处理器的云处理，则计算时间由 32 000 年变成了 2 个星期。

8.5 公钥的管理

公钥密码学使得人们有可能在不共享公共密钥的情况下仍然可以安全地进行通信。它也使人们有可能在不存在可信第三方的情况下为消息做签名。最后，利用签过名的消息摘要，接收方能够很容易地验证自己所接收到的消息的完整性。

然而，请等一等，我们这里还掩盖了一个问题：如果 Alice 和 Bob 彼此之间并不认识，那么，他们如何获得对方的公钥来开始通信呢？一种很容易想到的解决方案是将公钥放在自己的 Web 站点上，但是这种方案并不能工作，理由如下所述。假设 Alice 想要在 Bob 的 Web 站点寻找他的公钥，那么她该怎么办呢？首先，通过 Bob 的 URL，她用浏览器找到 Bob 主页的 DNS 地址，然后向该地址发送一个 GET 请求，如图 8-23 所示。不幸的是，Trudy 截获了这个请求，并且将一个伪造的主页发回给 Alice 作为应答，这个伪造的主页可能是 Bob 主页内容的一份副本，只不过其中 Bob 的公钥被替换成 Trudy 的公钥。当 Alice 现在用 E_T 加密她的第一条消息时，Trudy 解密并阅读该消息，然后用 Bob 的公钥重新进行加密，再发送给 Bob，而 Bob 根本不知道 Trudy 已经阅读了他所接收到的消息。更糟的是，Trudy 在为 Bob 重新加密消息之前还可以对消息做一些修改。很明显需要有某种机制来确保公钥交换的安全性。

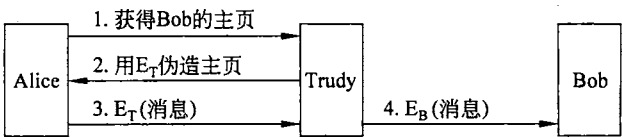


图 8-23 Trudy 颠覆公钥加密的一种方式